esprit
Se former autrement
HONORIS UNITED UNIVERSITIES





SkyCast:
Plateforme météorologique basée sur une architecture microservices.



année universitaire:2026/2027.

SkyCast



Yousfi Ilef



Amira NEFZI

Med Aziz Ferchichi



Yassmine Benslimene

Ahmed Omry

Yassine Ghedira

SOMMAIRE

1 Présentation générale du projet

2 Architecture globale

3 Organisation du groupe

4 Conception individuelle

5 Conclusion



1- Présentation générale du projet

Problème

Les données météo sont dispersées
(prévisions, qualité de l'air, alertes, historique).

Expérience fragmentée.



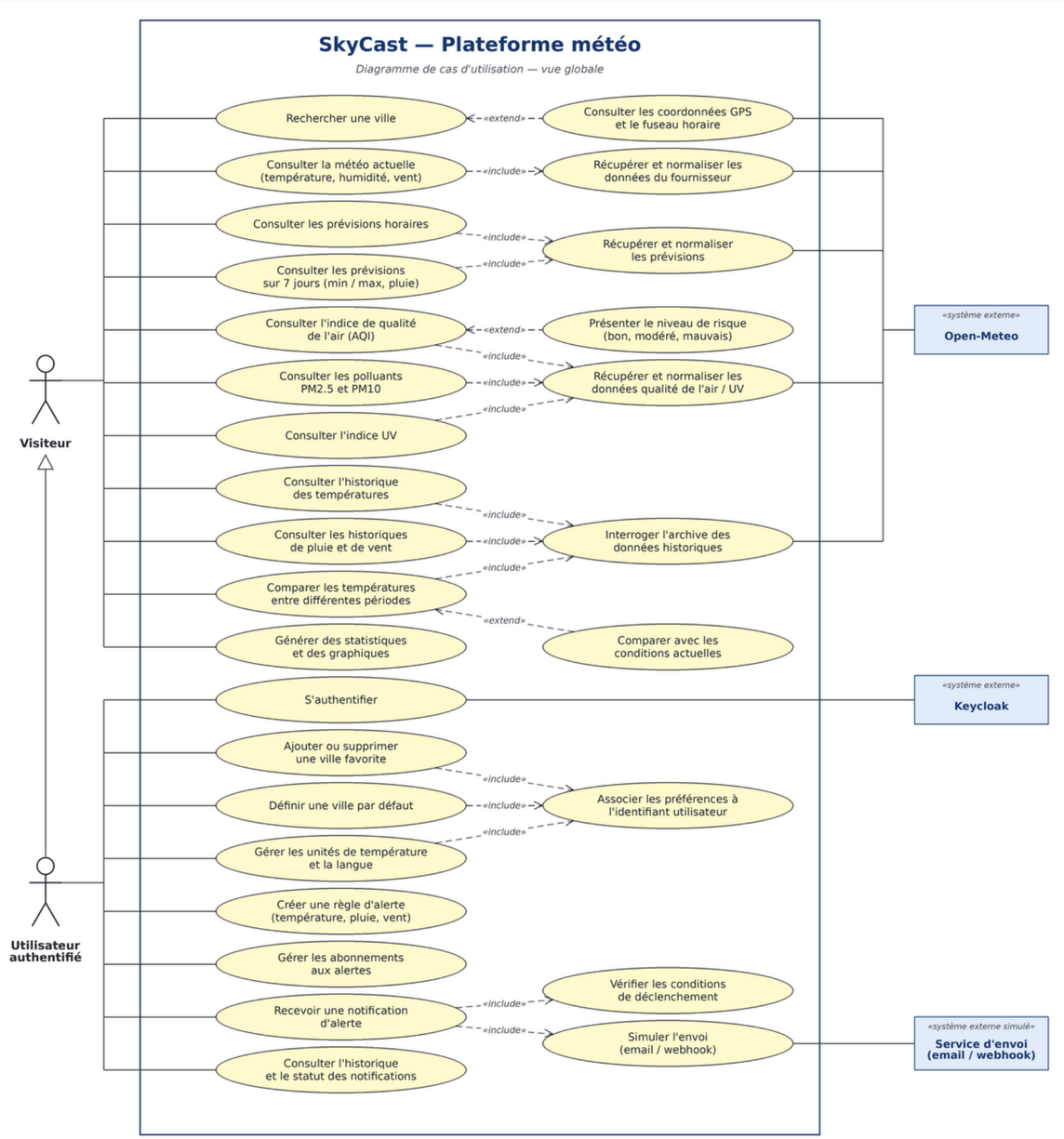
Solution : SkyCast

Une plateforme unique, centralisée
et personnalisée grâce à une architecture microservices.



Objectif : Modularité, évolutivité et indépendance des services

Diagramme de cas d’utilisation globale





2 -Architecture globale

Technologies choisies



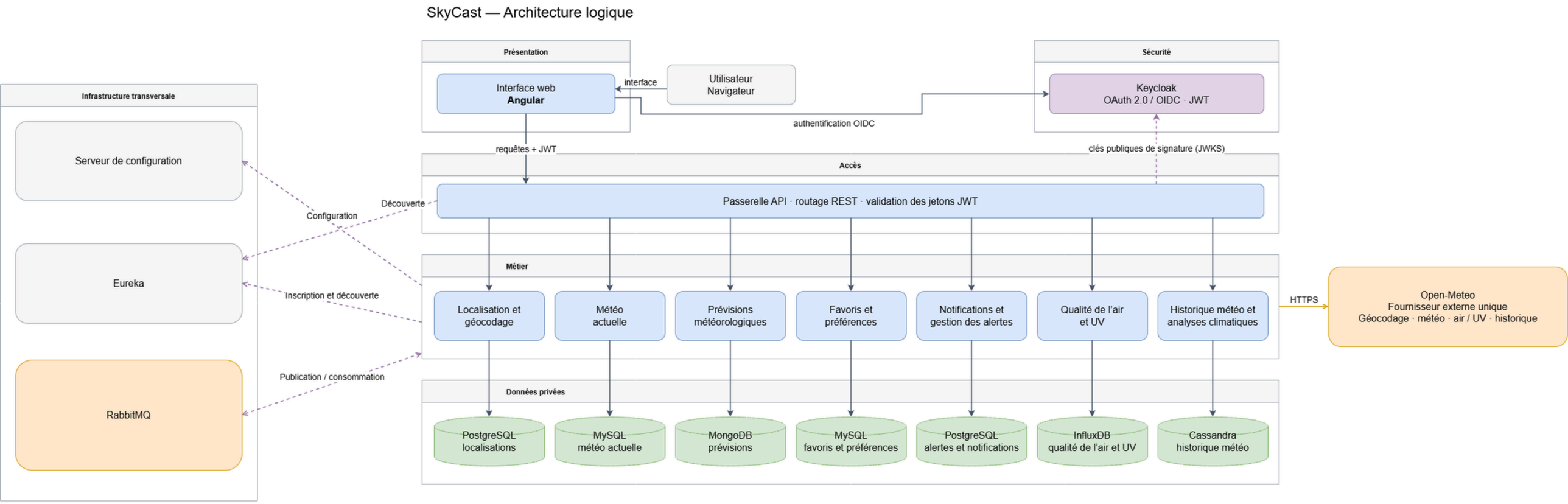
ANGULAR
MS1-MS7

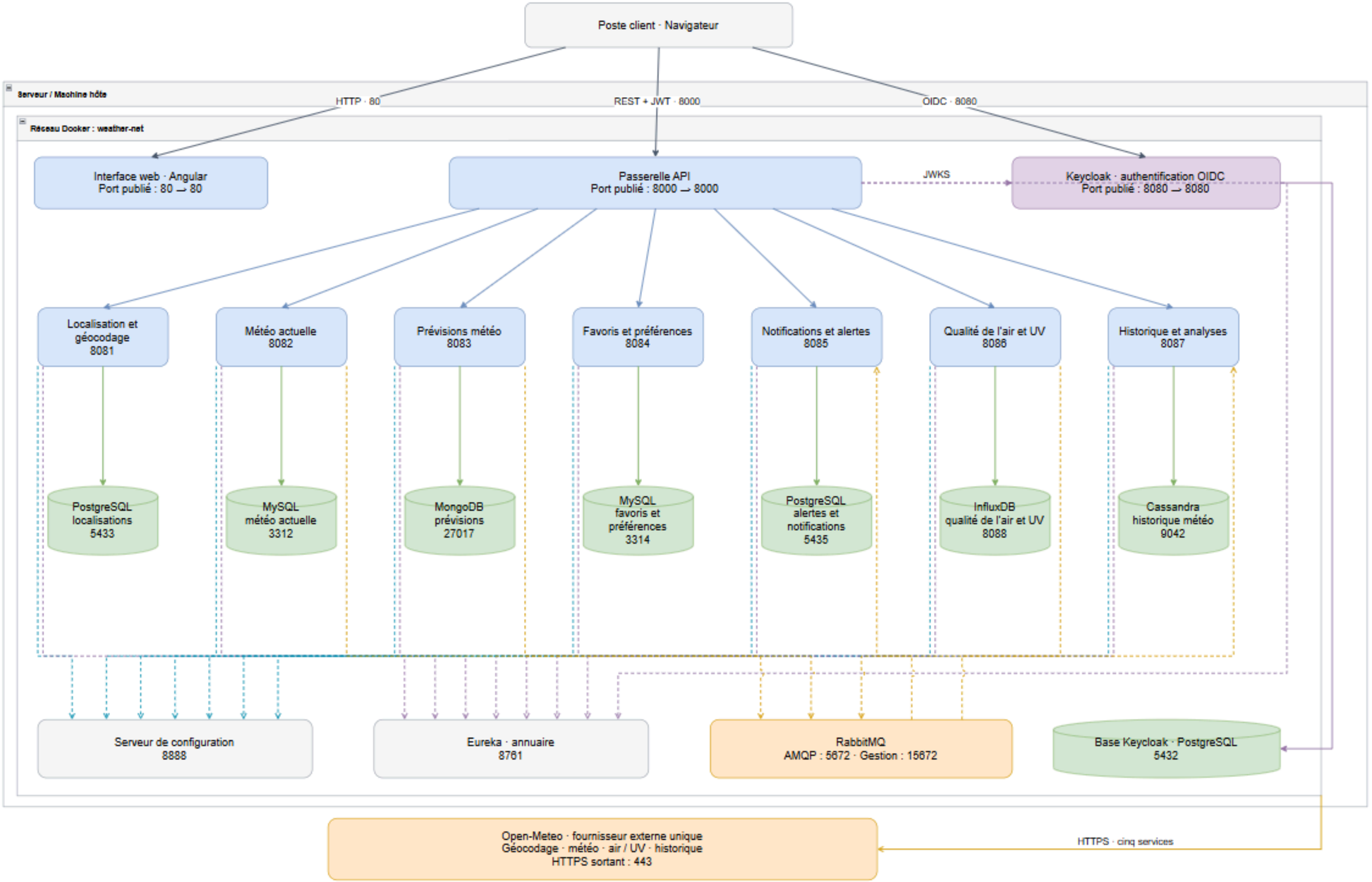


Spring Boot
MS1-MS6



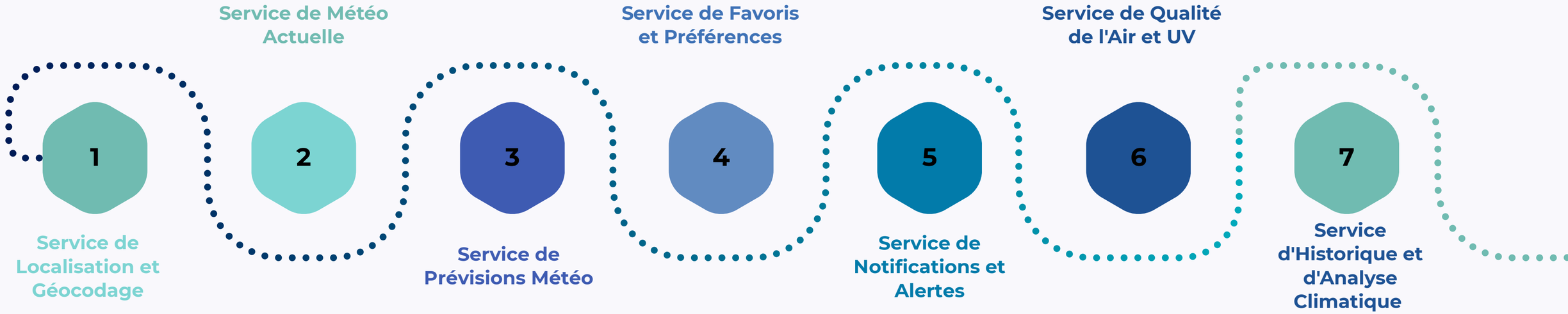
Django
MS7

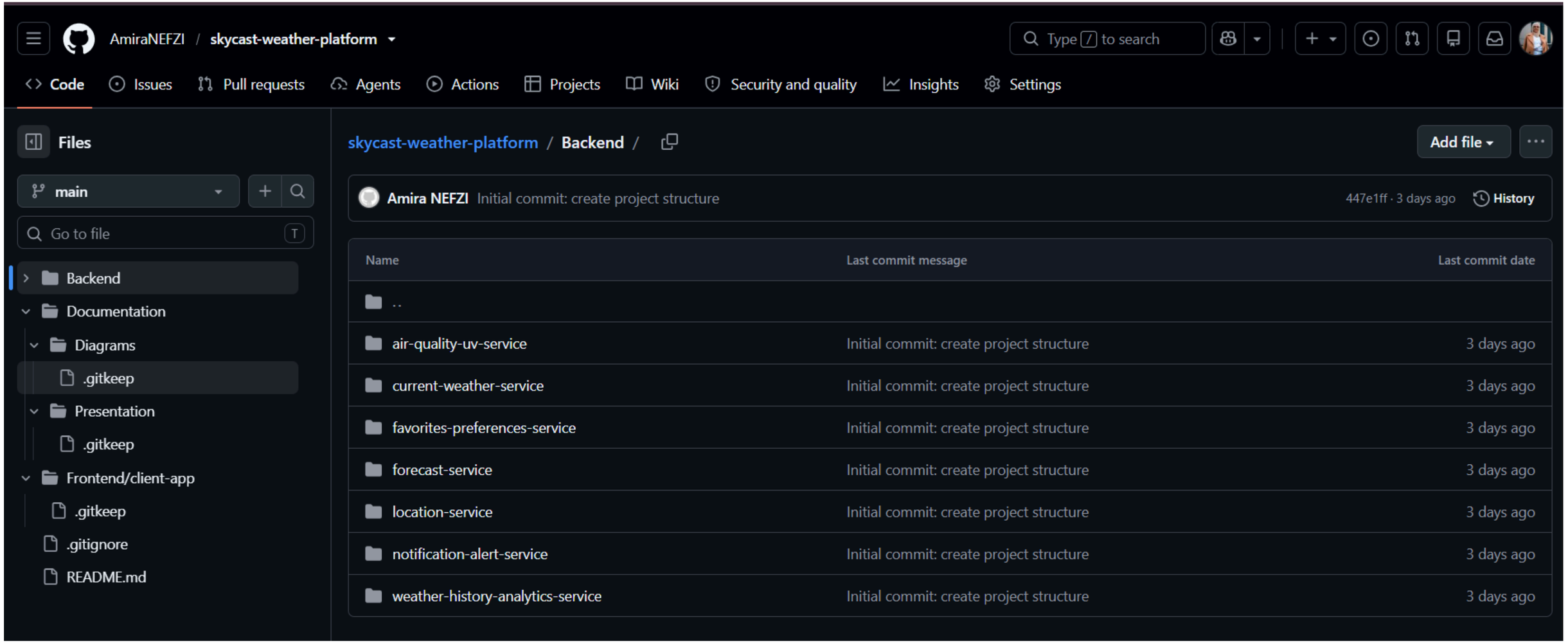

SkyCast — Architecture logique
Présentation
Interface web
Angular
interface
Utilisateur
Navigateur
Sécurité
Keycloak
OAuth 2.0 / OIDC · JWT
authentification OIDC
requêtes + JWT
clés publiques de signature (JWKS)
Accès
Passerelle API · routage REST · validation des jetons JWT
Infrastructure transversale
Serveur de configuration
Eureka
RabbitMQ
Découverte
Configuration
Inscription et découverte
Publication / consommation
Métier
Localisation et géocodage
Météo actuelle
Prévisions météorologiques
Favoris et préférences
Notifications et gestion des alertes
Qualité de l'air et UV
Historique météo et analyses climatiques
HTTPS
Open-Meteo
Fournisseur externe unique
Géocodage · météo · air / UV · historique
Données privées
PostgreSQL
localisations
MySQL
météo actuelle
MongoDB
prévisions
MySQL
favoris et préférences
PostgreSQL
alertes et notifications
InfluxDB
qualité de l'air et UV
Cassandra
historique météo


Poste client · Navigateur
Serveur / Machine hôte
HTTP · 80
REST + JWT · 8000
OIDC · 8080
Réseau Docker : weather-net
Interface web · Angular
Port publié : 80 → 80
Passerelle API
Port publié : 8000 → 8000
JWKS
Keycloak · authentification OIDC
Port publié : 8080 → 8080
Localisation et géocodage 8081
Météo actuelle 8082
Prévisions météo 8083
Favoris et préférences 8084
Notifications et alertes 8085
Qualité de l'air et UV 8086
Historique et analyses 8087
PostgreSQL localisations 5433
MySQL météo actuelle 3312
MongoDB prévisions 27017
MySQL favoris et préférences 3314
PostgreSQL alertes et notifications 5435
InfluxDB qualité de l'air et UV 8088
Cassandra historique météo 9042
Serveur de configuration 8888
Eureka · annuaire 8761
RabbitMQ
AMQP : 5672 · Gestion : 15672
Base Keycloak · PostgreSQL 5432
Open-Meteo · fournisseur externe unique
Géocodage · météo · air / UV · historique
HTTPS sortant : 443
HTTPS · cinq services



3 -Organisation du groupe


1
Service de Localisation et Géocodage
2
Service de Météo Actuelle
3
Service de Prévisions Météo
4
Service de Favoris et Préférences
5
Service de Notifications et Alertes
6
Service de Qualité de l'Air et UV
7
Service d'Historique et d'Analyse Climatique


AmiraNEFZI / skycast-weather-platform
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file T
Backend
Documentation
Diagrams
.gitkeep
Presentation
.gitkeep
Frontend/client-app
.gitkeep
.gitignore
README.md
skycast-weather-platform / Backend /
Add file
Amira NEFZI Initial commit: create project structure
447e1ff · 3 days ago
History
Name
Last commit message
Last commit date
..
air-quality-uv-service Initial commit: create project structure 3 days ago
current-weather-service Initial commit: create project structure 3 days ago
favorites-preferences-service Initial commit: create project structure 3 days ago
forecast-service Initial commit: create project structure 3 days ago
location-service Initial commit: create project structure 3 days ago
notification-alert-service Initial commit: create project structure 3 days ago
weather-history-analytics-service Initial commit: create project structure 3 days ago

SkyCast

Planning Prévisionnel

Une organisation claire pour réussir notre projet SkyCast

S1 — Semaine 1

- Analyse des besoins
- Conception initiale de l'architecture

S2 — Semaine 2

- Conception des bases de données
- Modélisation UML

S3 — Semaine 3

- Développement des microservices (MS1, MS2, MS3)

S5 — Semaine 5

- Développement des microservices (MS4, MS5, MS6)

S6 — Semaine 6

- Développement des microservices (MS7) et intégration

S7 — Semaine 7

- Tests fonctionnels et d'intégration
- Correction des anomalies

S8 — Semaine 8

- Finalisation, documentation et préparation de la présentation

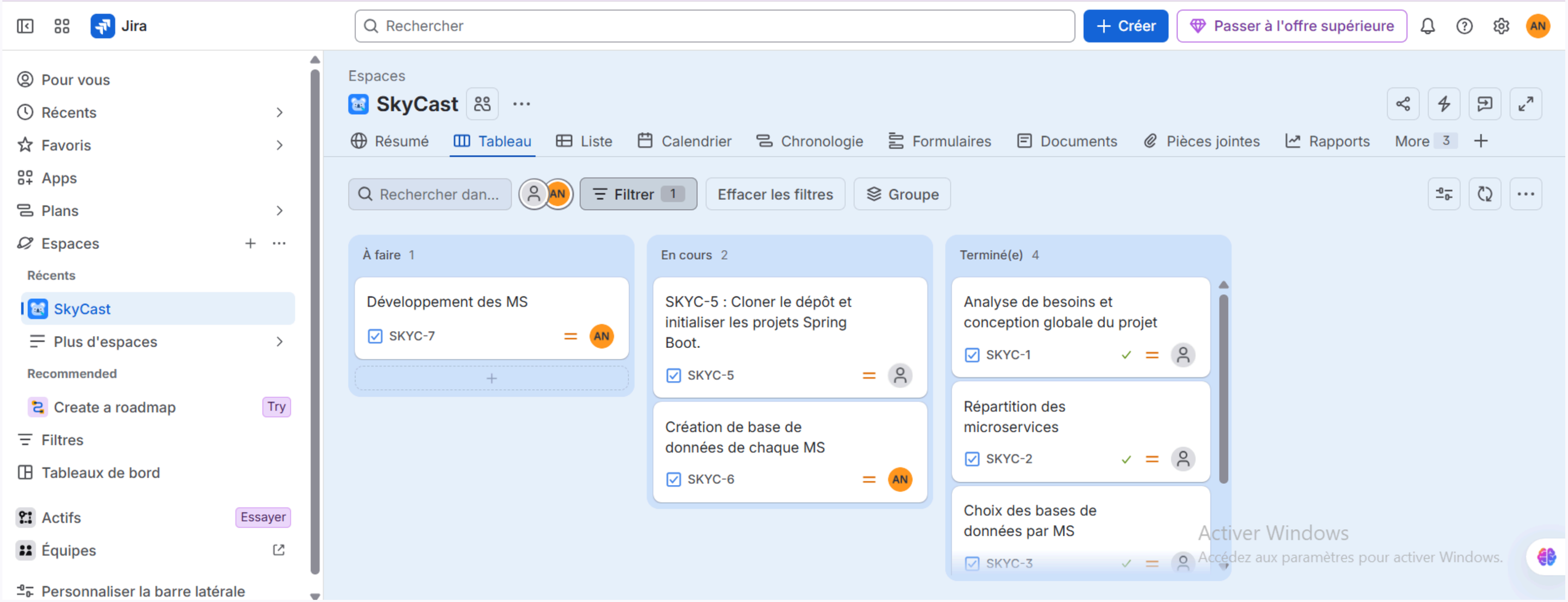
S9 — Semaine 9

- Présentation finale du projet

Objectif

Assurer une progression organiséé sur 8 semaines, une bonne coordination entre les membres et une intégration cohérente de tous les services.

Gestion du projet avec jira



SkyCast

Principaux risques et solutions

Anticiper les défis pour assurer la réussite de notre projet

Risques identifiés		Solutions proposées
Problèmes d'intégration entre les microservices	→	Définir des API communes et tester régulièrement les échanges REST.
Indisponibilité d'Open-Meteo	→	Gérer les erreurs et utiliser les dernières données disponibles lorsque possible.
Incohérence des données entre les services	→	Normaliser les données et respecter les contrats d'API.
Problèmes de sécurité	→	Utiliser Keycloak, les jetons JWT et la validation au niveau de la passerelle API.
Retards dans le développement	→	Répartir clairement les responsabilités et suivre l'avancement régulièrement.
Erreurs lors de l'intégration	→	Effectuer des tests unitaires et des tests d'intégration avant la livraison.

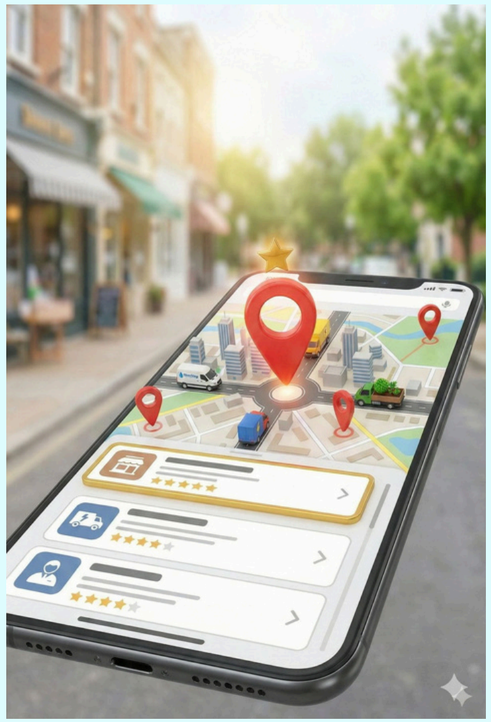
Objectif

Anticiper les difficultés techniques et organisationnelles afin de garantir la fiabilité et la stabilité de SkyCast.



4 - Conception individuelle

MS1 — Service de Localisation et de Géocodage

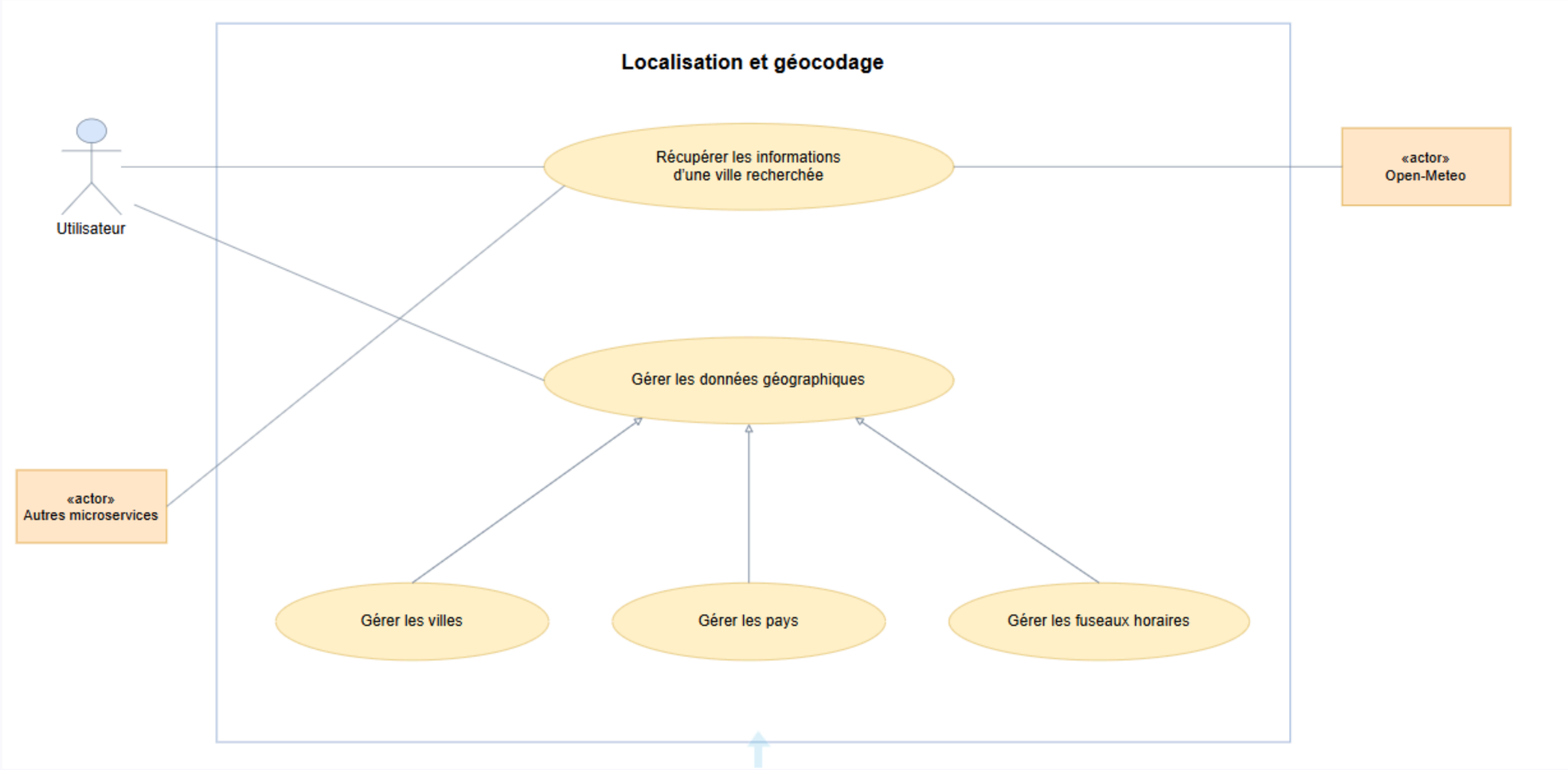


Objectif

Gérer les informations géographiques des villes, notamment leurs coordonnées GPS (latitude et longitude) et les fuseaux horaires.

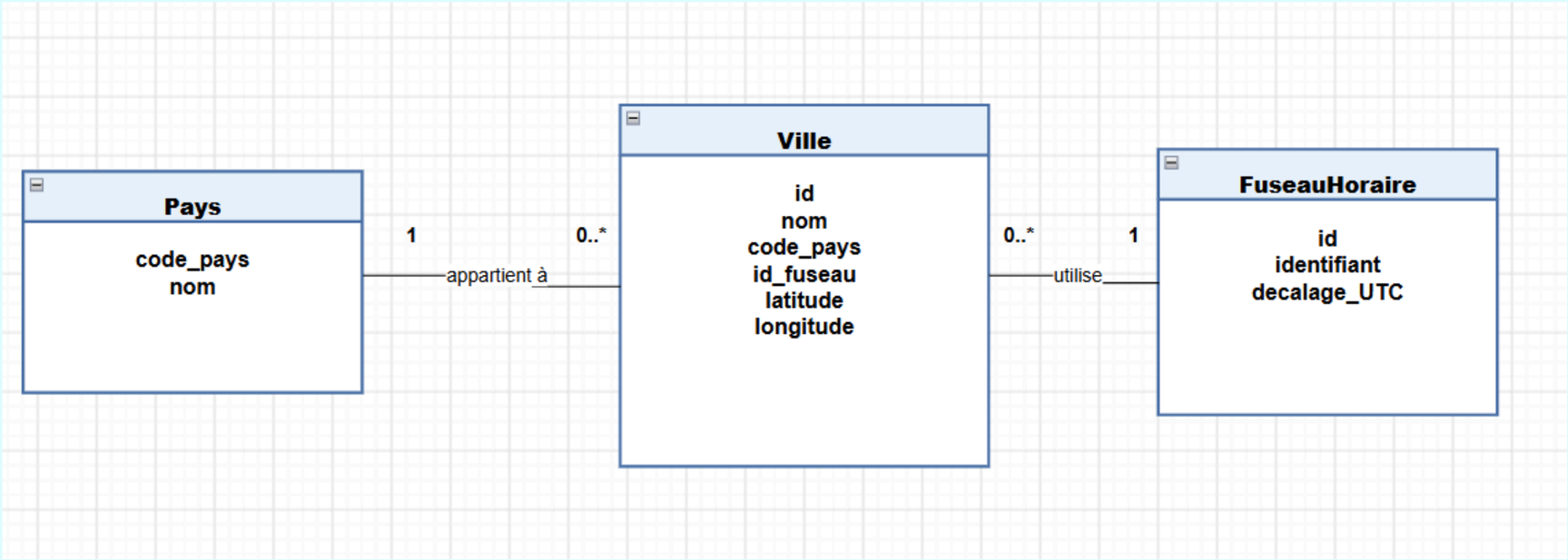
MS1 — Service de Localisation et de Géocodage

Fonctionnalités principales



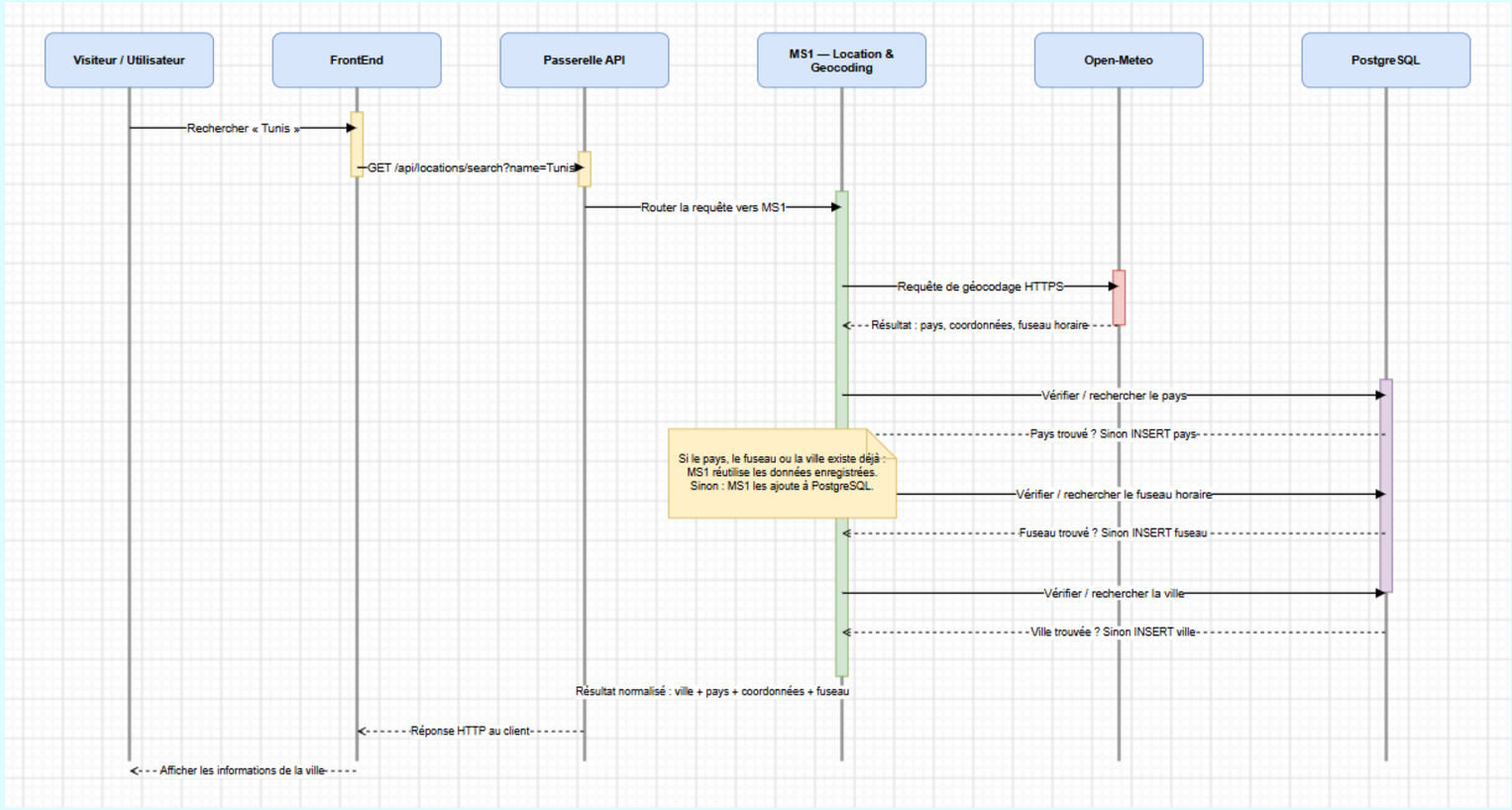
MS1 — Service de Localisation et de Géocodage

Conception



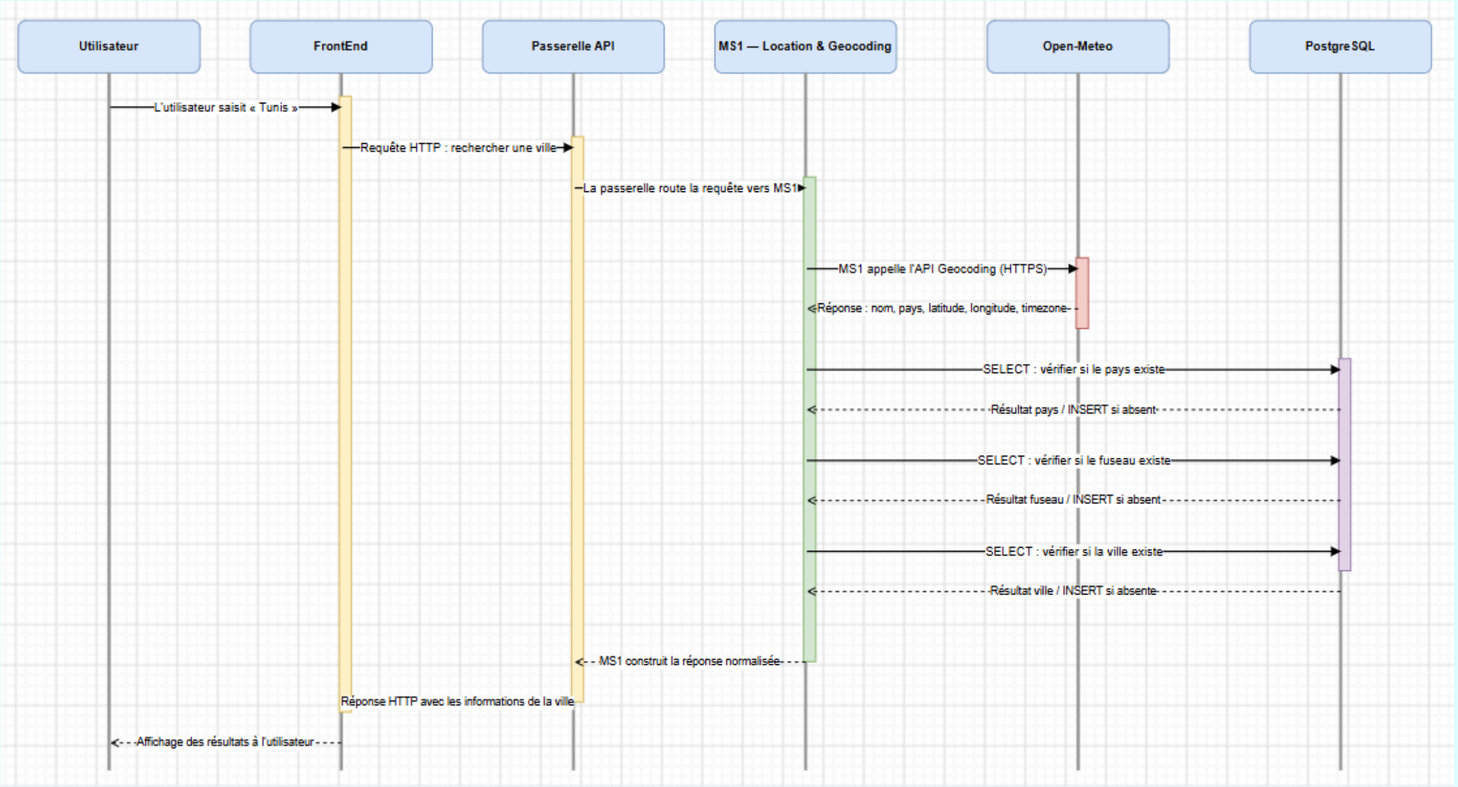
MS1 — Service de Localisation et de Géocodage

Communication Synchrone et asynchrone

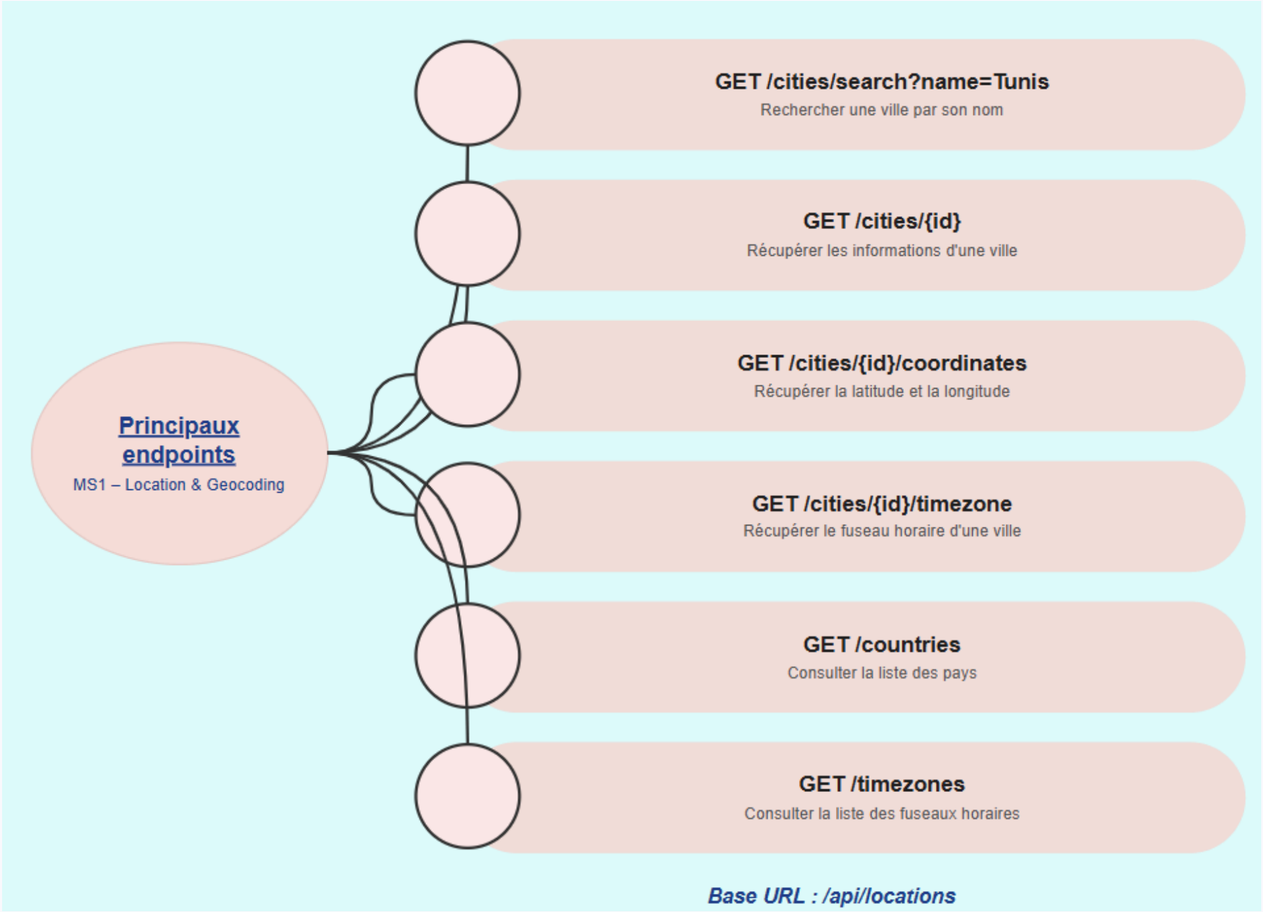


MS1 — Service de Localisation et de Géocodage

Communication Synchrone et synchrone



MS1 — Service de Localisation et de Géocodage



MS1 — Service de Localisation et de Géocodage

MS1 Location & Geocoding Service

API Endpoints

GET	/api/locations/cities/search?name={name}	Search for a city by name
GET	/api/locations/cities/{id}	Get city details
GET	/api/locations/cities/{id}/coordinates	Get latitude and longitude
GET	/api/locations/cities/{id}/timezone	Get timezone information
GET	/api/locations/countries	Get list of countries
GET	/api/locations/timezones	Get list of timezones

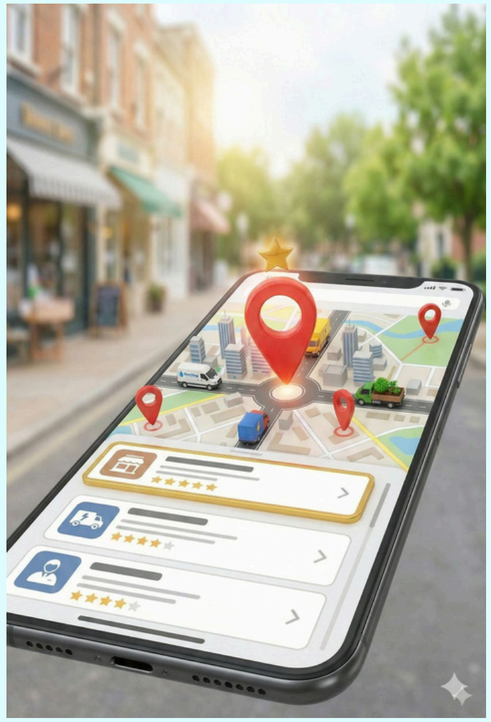
Response 200 OK JSON

```
{
  "id": 1,
  "name": "Tunis",
  "country": "Tunisia",
  "latitude": 36.8065,
  "longitude": 10.1815,
  "timezone": "Africa/Tunis"
}
```

Status Codes

200	404	400	500
Success	City not found	Invalid request	Server error

MS2 — Service de Météo actuelle

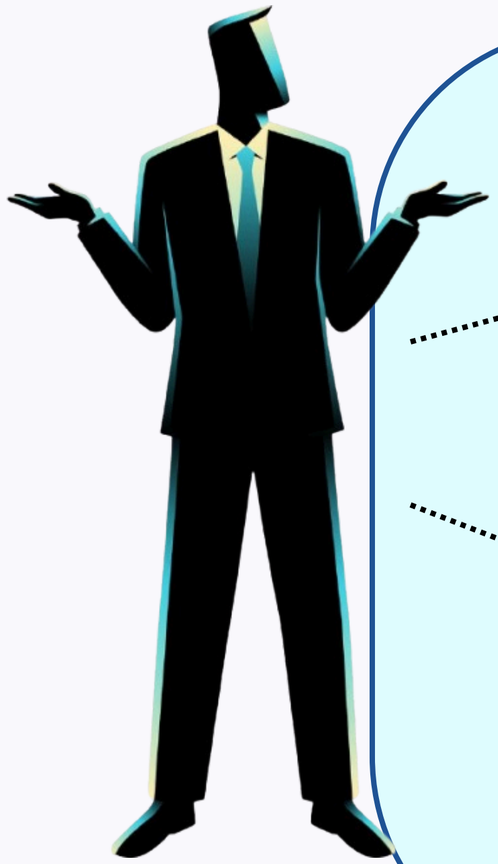


Objectif

Permettre à l’utilisateur de connaître en temps réel la météo actuelle d’une ville.

MS2 — Service de Météo actuelle

Fonctionnalités principales



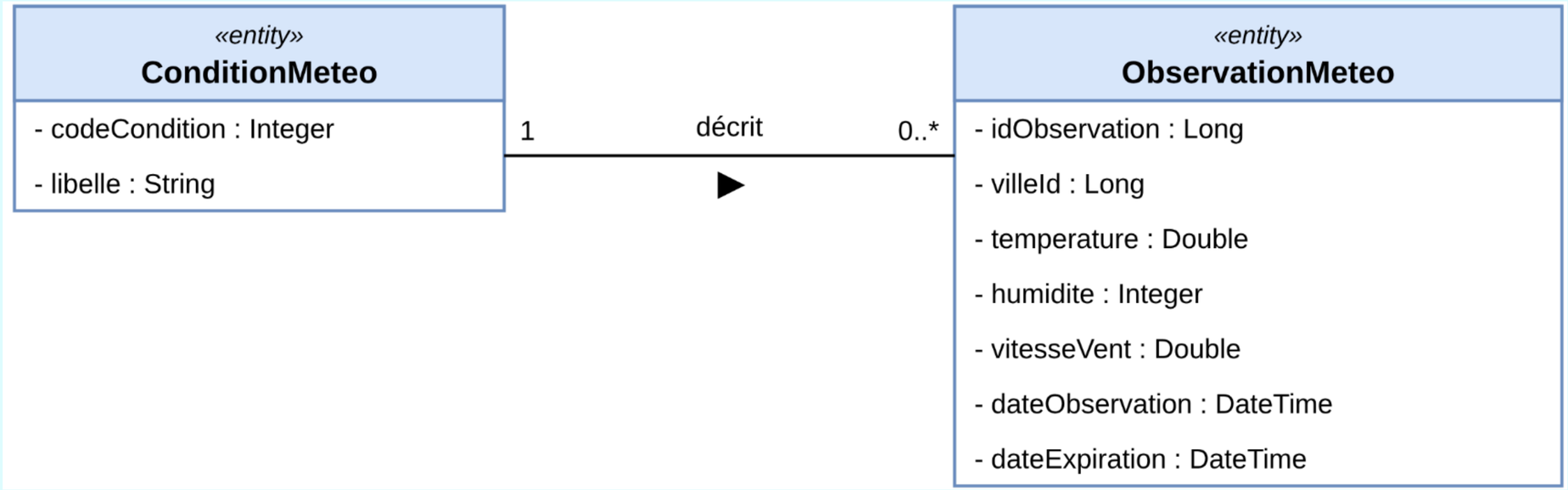


Consulter la température, l'humidité et la vistesse du vent.

Consulter les conditions météorologiques actuelles

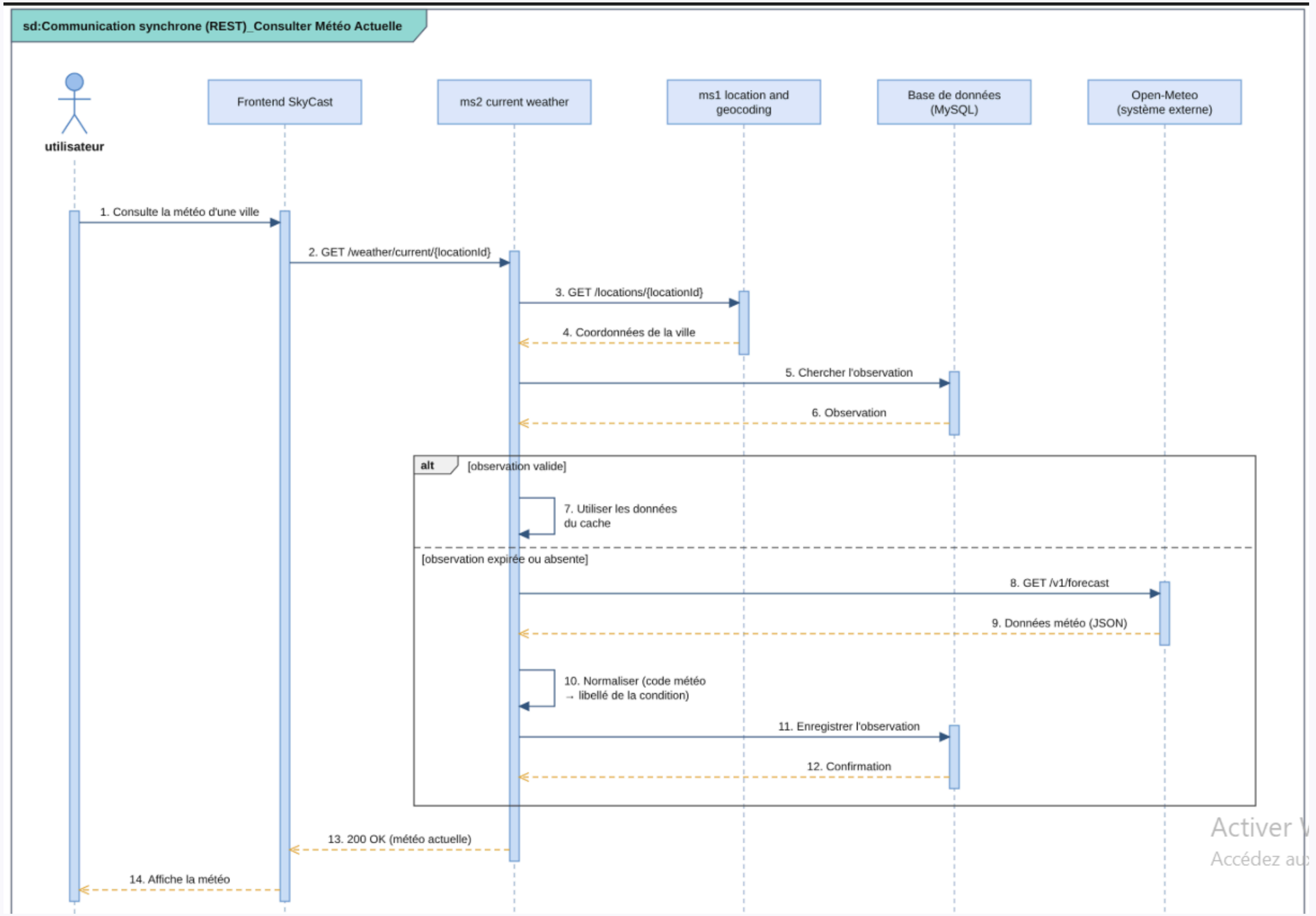
MS2 — Service de Météo actuelle

Conception

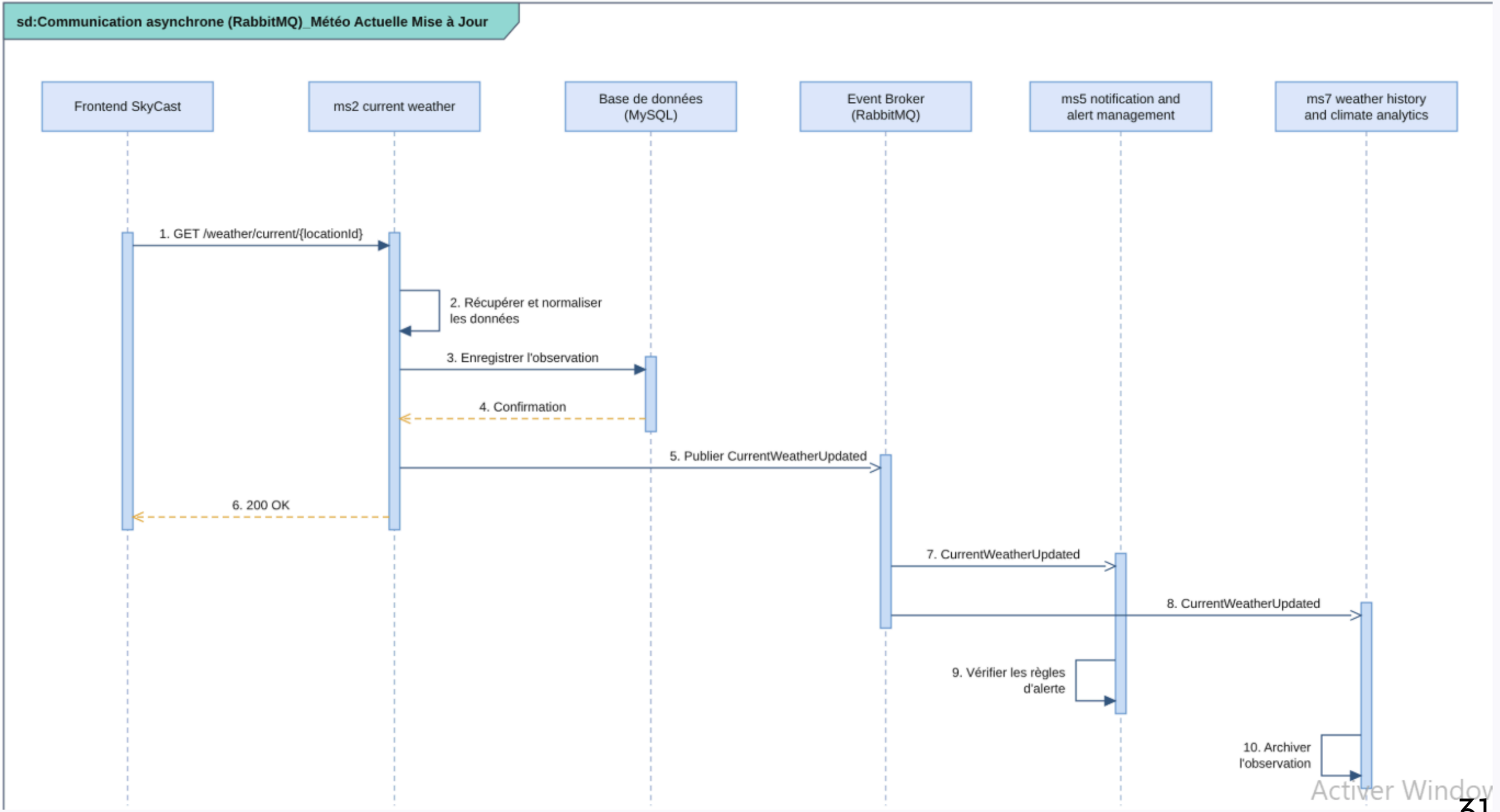


MS2 — Service de Météo actuelle

Communication Synchrone et asynchrone

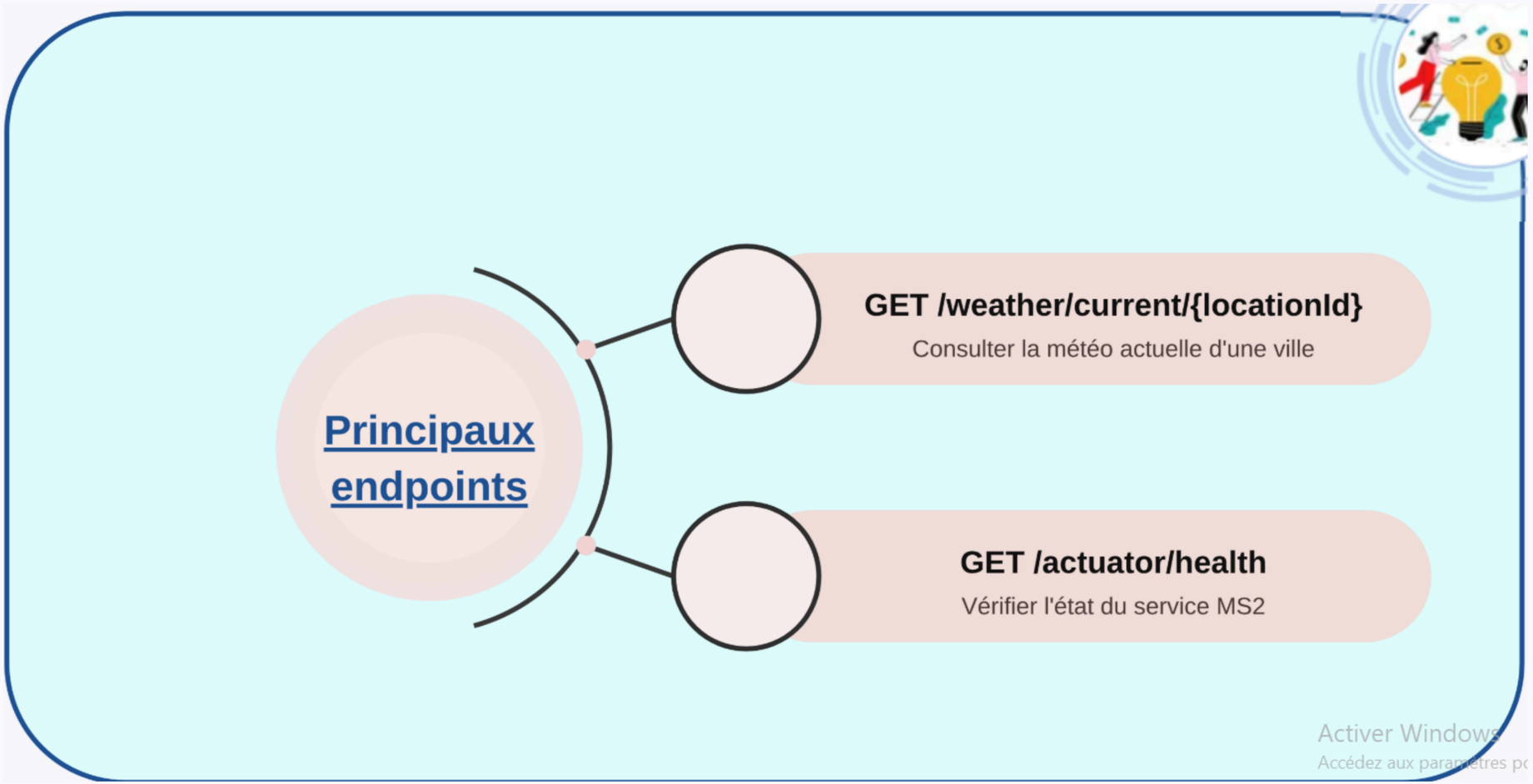


Activer
Accédez au



Activer Windows

MS2 — Service de Météo actuelle



Activer Windows
Accédez aux paramètres pc

Contrat d'API — MS2 Current Weather

Endpoint principal exposé par le service

GET `/weather/current/{locationId}`

Rôle

Retourne la météo actuelle
d'une ville

Accès

Visiteur et utilisateur
(jeton JWT facultatif)

Paramètre

locationId (Long)
Identifiant de la ville dans MS1

Cache

Observation réutilisée tant que
dateExpiration n'est pas dépassée

Réponse 200 OK

JSON

```
{
  "villeId": 12,
  "nomVille": "Tunis",
  "temperature": 24.5,
  "humidite": 61,
  "vitesseVent": 14.2,
  "condition": "Ciel dégagé",
  "dateObservation": "2026-11-11T10:00:00Z",
  "dateExpiration": "2026-11-11T10:15:00Z"
}
```

Codes d'erreur

400	404	503
Identifiant invalide	Ville inconnue	Source indisponible

Activer Windows
Accédez aux paramè

Contrat d'API — Échanges de MS2

Appels sortants, événement publié et supervision

Appels sortants

MS2 interroge…

MS1

GET /locations/{locationId}

Coordonnées et fuseau horaire de la ville

Open-Meteo

GET /v1/forecast

Météo brute : température, humidité, vent, code météo

Événement publié

Canal asynchrone (RabbitMQ)

CurrentWeatherUpdated

Quand

Après chaque actualisation d'une observation

Consommé par

MS5 · alertes
MS7 · historique

Contenu

villeId, temperature, humidite, vitesseVent, condition, dateObservation

Endpoint technique

Supervision du service

GET **/actuator/health**

Rôle

Indique que MS2 est actif

Utilité

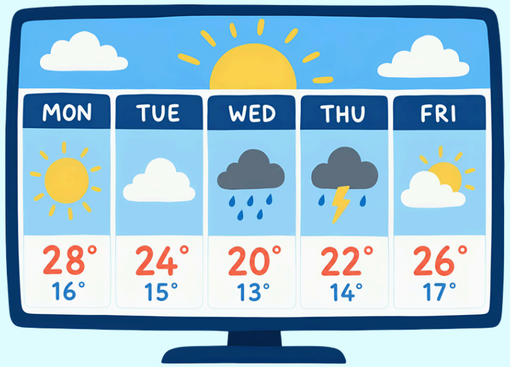
Supervision et registre Eureka

Accès

Technique (non métier)

```
{ "status": "UP" }
```

MS3 — Service de Prévisions Météo

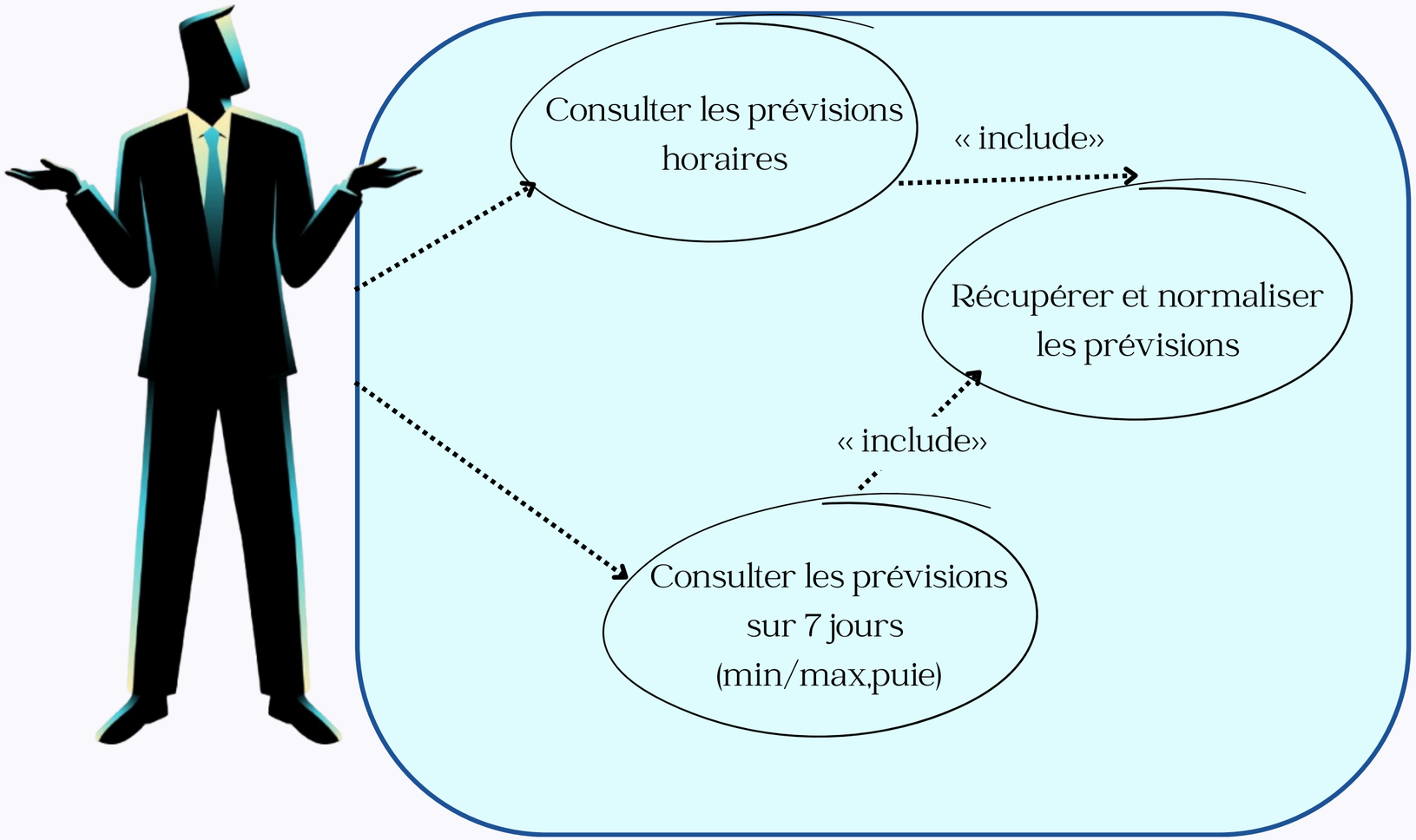


Objectif

Fournir aux utilisateurs les prévisions météo (heure par heure et sur 7 jours) pour une ville donnée, en récupérant les données du fournisseur externe Open-Meteo et en les stockant dans sa base MongoDB

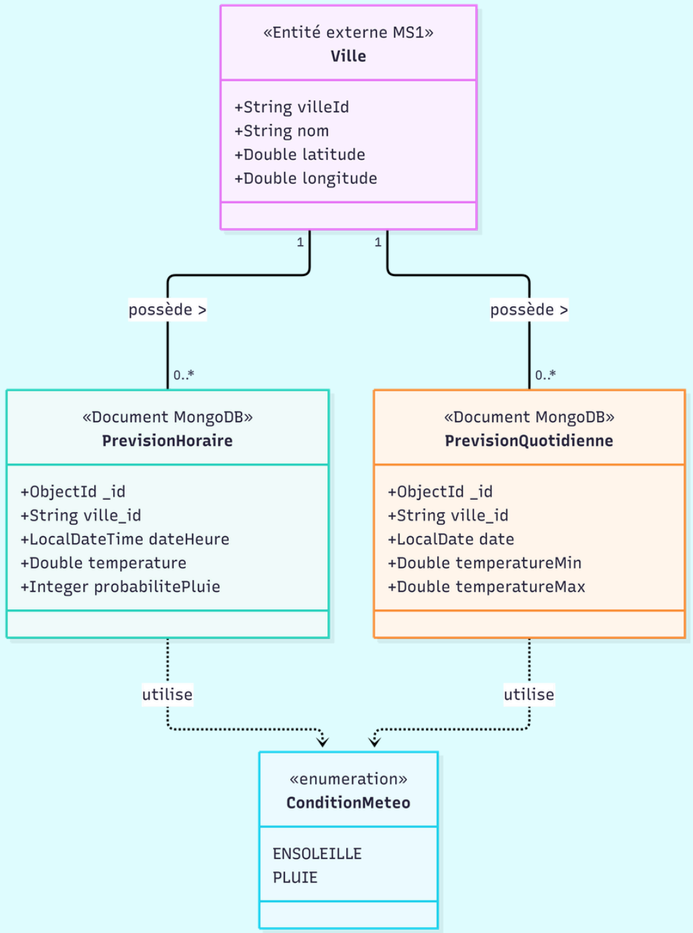
MS3 — Service de Prévisions Météo

Fonctionnalités principales



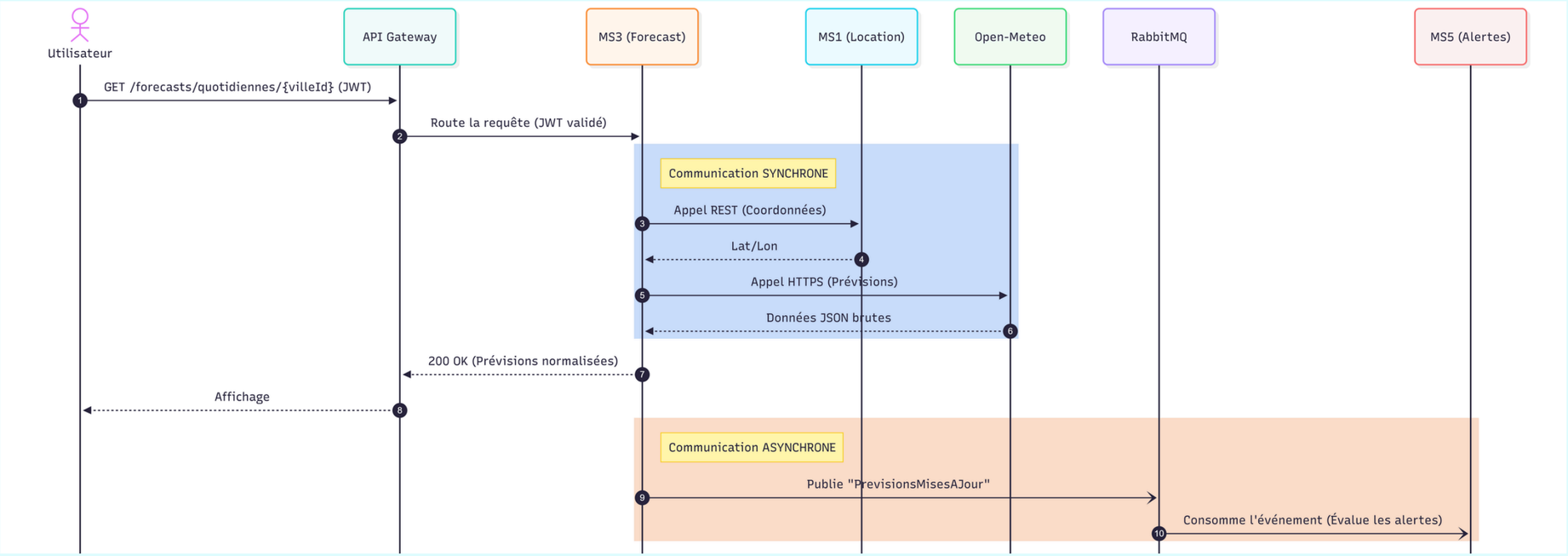
MS3 — Service de Prévisions Météo

Diagramme de Classes



MS3 — Service de Prévisions Météo

Communication Synchrone et asynchrone



MS3 — Service de Prévisions Météo



Principaux endpoints

- **GET /api/forecasts /quotidiennes/{villeId}**
- **POST/api/forecasts /sync/{villeId}**
- **GET /api/forecasts /horaires/{villeId}**

Contrat d'API — MS3 Prévisions Météo

Endpoints principaux exposés par le service

GET `/api/forecasts/horaires/{villeId}`

GET `/api/forecasts/quotidiennes/{villeId}`

POST `/api/forecasts/sync/{villeId}`

Rôle
Retourne les prévisions
horaires et sur 7 jours d'une ville

Accès
Visiteur et utilisateur
(jeton JWT facultatif) · sync : technique

Paramètre
villeId (String)
Identifiant de la ville dans MS1

Cache
Prévisions réutilisées tant que
la dernière synchronisation est récente

Réponse 200 OK

JSON

GET quotidiennes

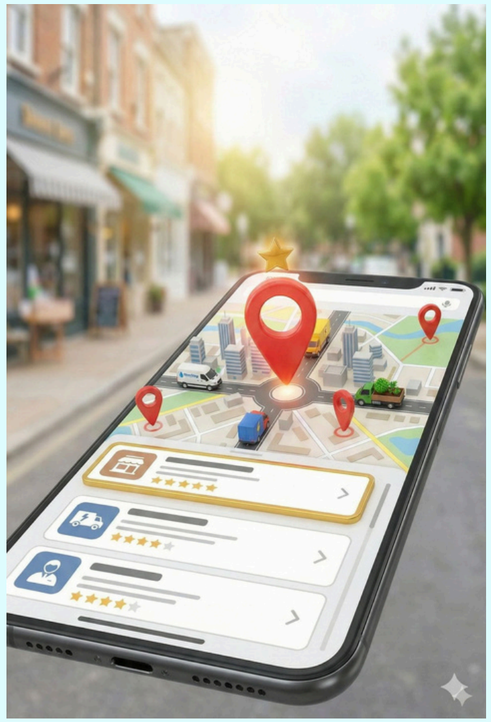
```
{
  "ville_id": "1",
  "previsions": [
    { "date": "2026-11-12",
      "temperatureMin": 14.2,
      "temperatureMax": 23.8,
      "condition": "PLUIE" }, ...
  ]
}
```

Codes d'erreur

400	404	503
Identifiant invalide	Ville inconnue	Source indisponible

POST /sync : 401/403 si non autorisé

MS4 — Favorites & User Preferences

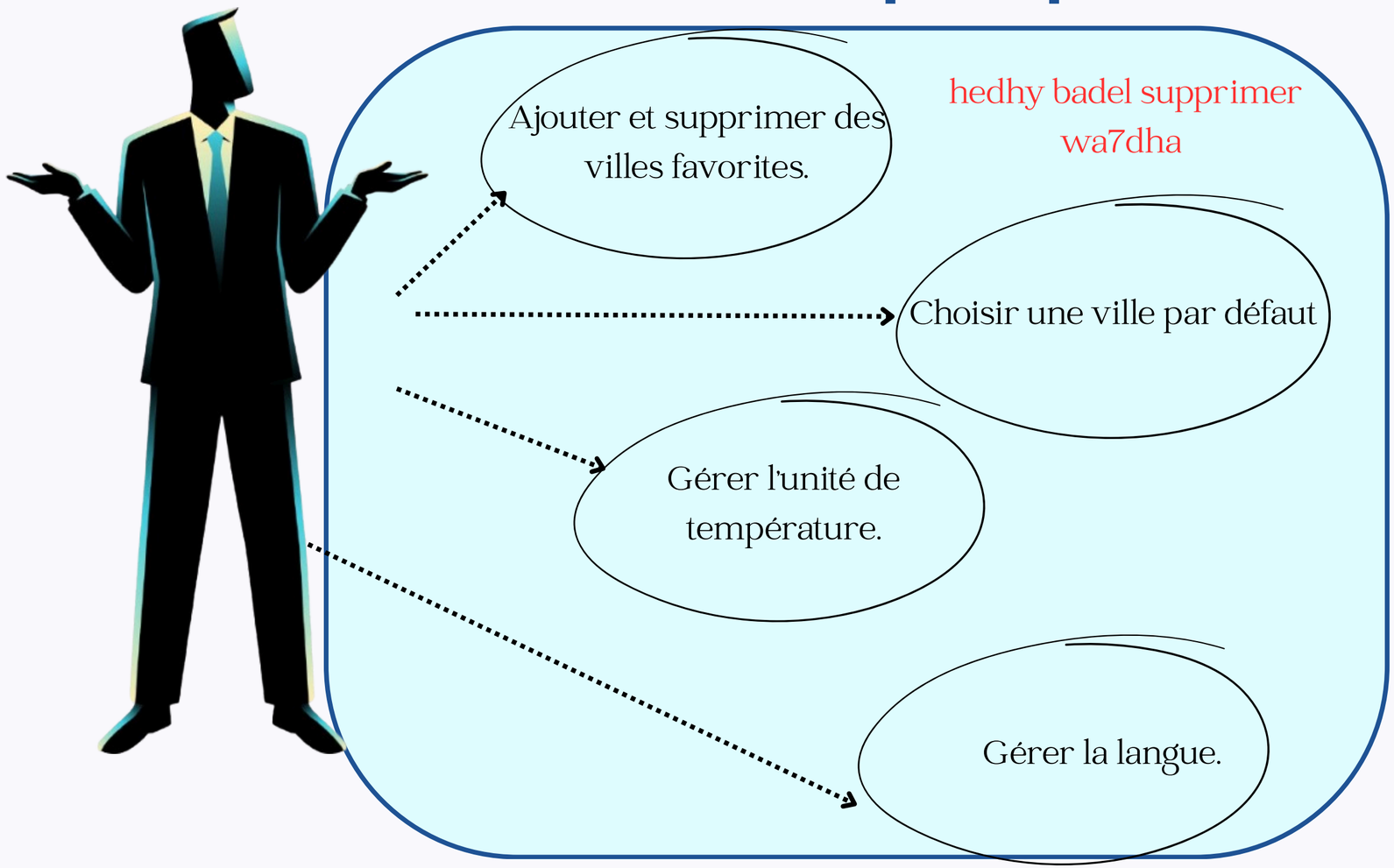


Objectif

Gérer les villes favorites et les préférences personnelles des utilisateurs.

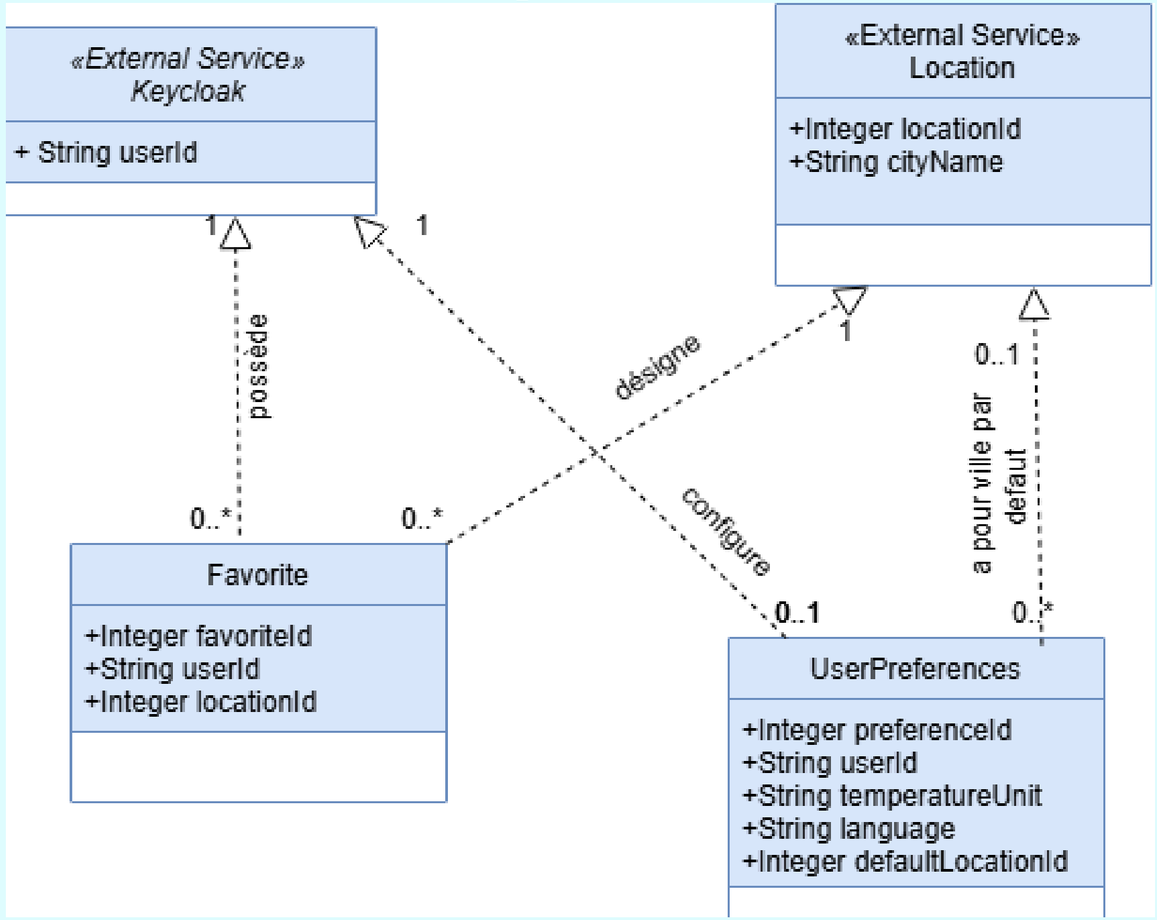
MS4 — Favorites & User Preferences

Fonctionnalités principales



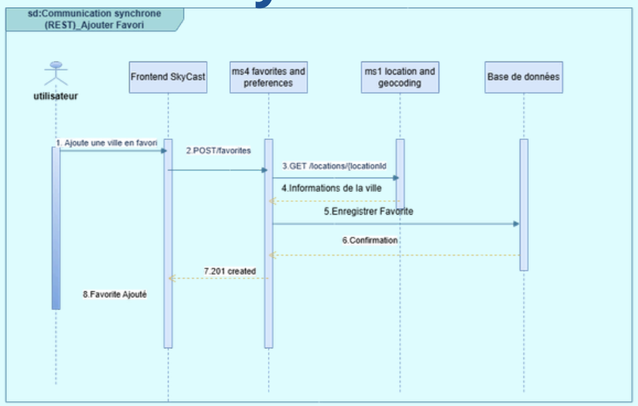
MS4 — Favorites & User Preferences

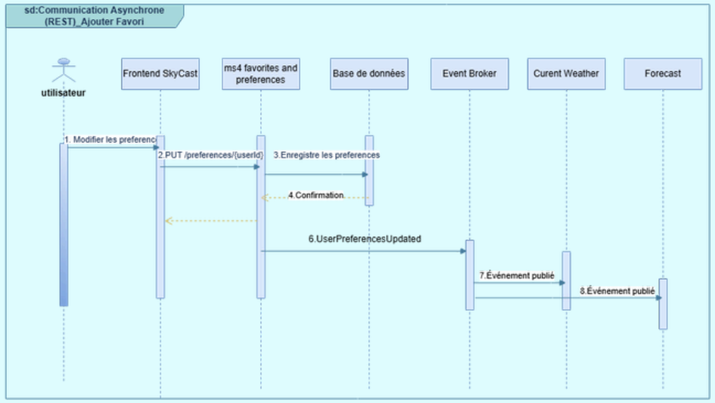
Conception

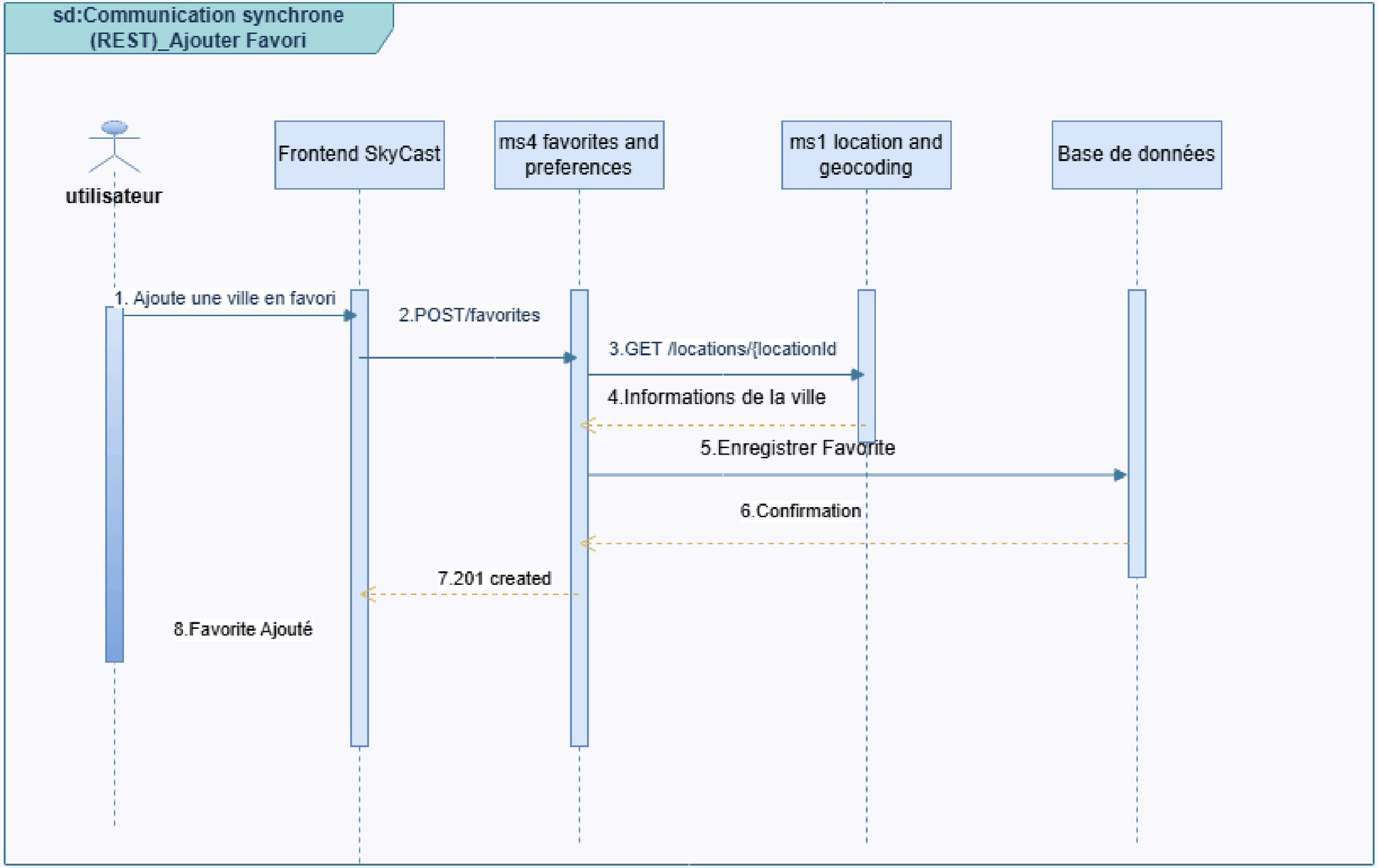


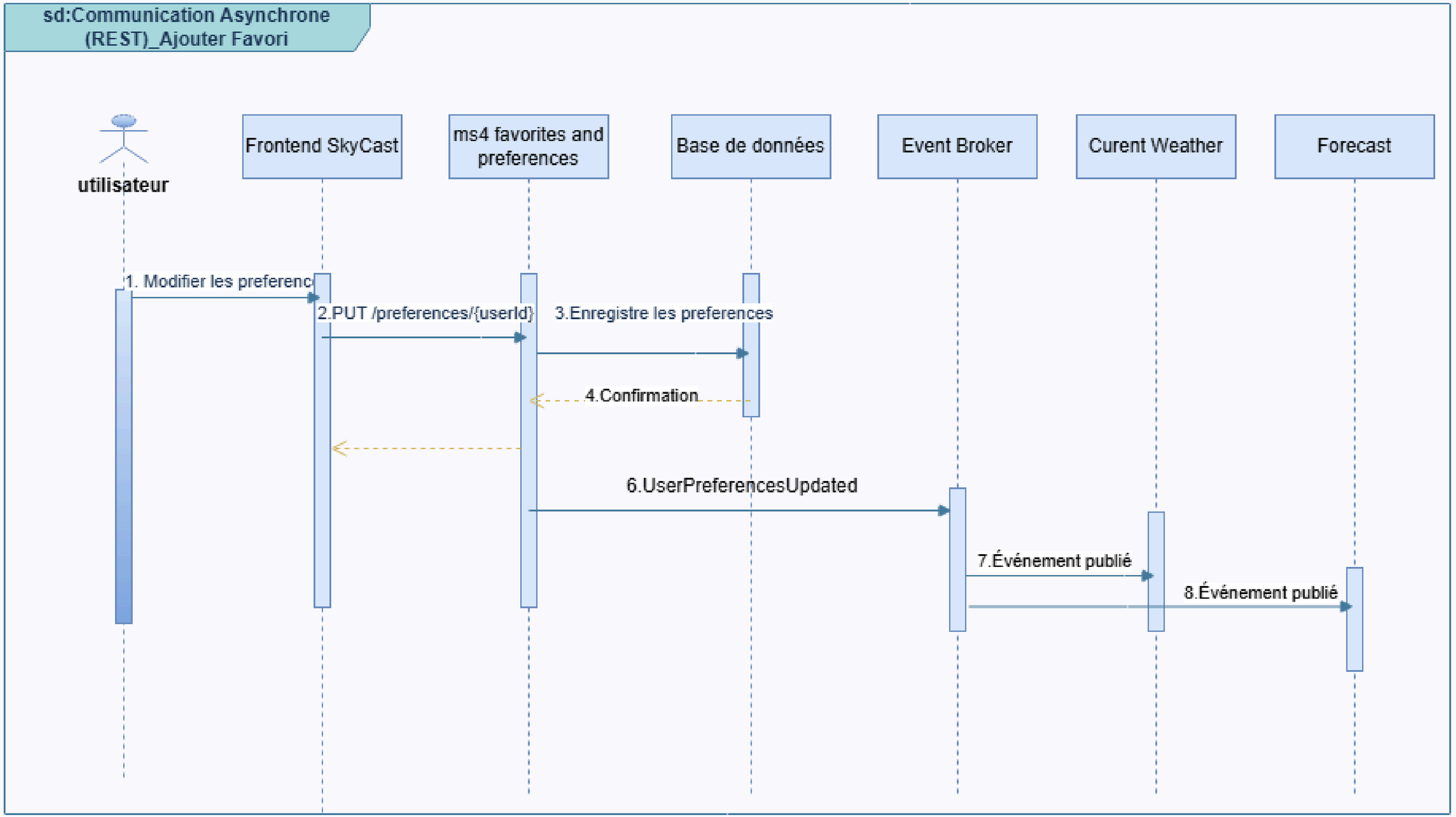
MS4 — Favorites & User Preferences

Communication Synchrone et asynchrone

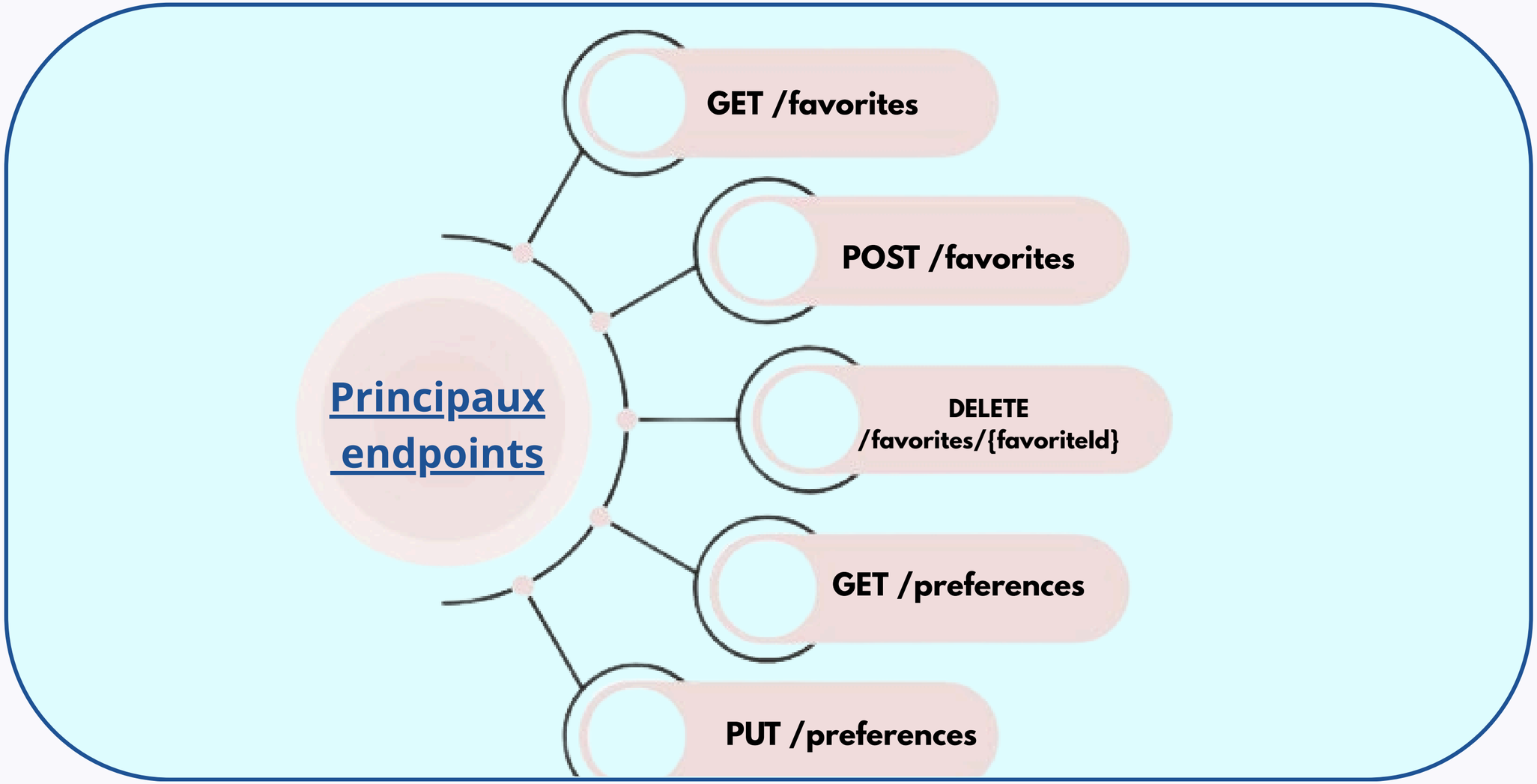





sd:Communication synchrone (REST)_Ajouter Favori
utilisateur
Frontend SkyCast
ms4 favorites and preferences
ms1 location and geocoding
Base de données
1. Ajoute une ville en favori
2.POST/favorites
3.GET /locations/{locationId
4.Informations de la ville
5.Enregistrer Favorite
6.Confirmation
7.201 created
8.Favorite Ajouté


sd:Communication Asynchrone (REST)_Ajouter Favori
utilisateur
Frontend SkyCast
ms4 favorites and preferences
Base de données
Event Broker
Curent Weather
Forecast
1. Modifier les preferenc
2.PUT /preferences/{userId}
3.Enregistre les preferences
4.Confirmation
6.UserPreferencesUpdated
7.Événement publié
8.Événement publié

MS4 — Favorites & User Preferences



MS4 — Favorites & User Preferences

Contrat d'API — MS4 Favorites & User Preferences

Endpoint principal : ajouter un favori

POST `/favorites`

Rôle	**Ajoute une ville aux favoris** de l'utilisateur connecté
Accès	**Utilisateur connecté** jeton JWT Keycloak obligatoire
Corps	**locationId (Long)** identifiant de la ville dans MS1
Vérification	**MS4 interroge MS1** appel REST synchrone pour confirmer que la ville existe

Réponse 201 Created

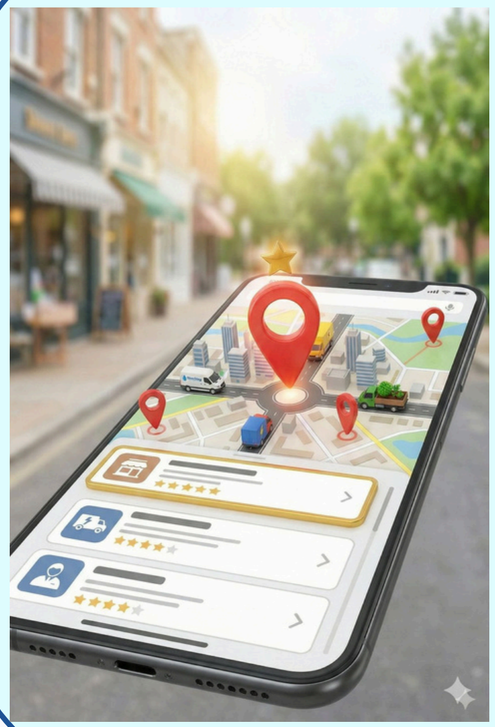
JSON

```
{
  "favoriteId": 5,
  "locationId": 12
}
```

Codes d'erreur

400 Corps invalide	**401** Non authentifié	**404** Ville inconnue
409 Déjà en favori	**503** MS1 indisponible	

MS5 — Notifications et gestion des alertes

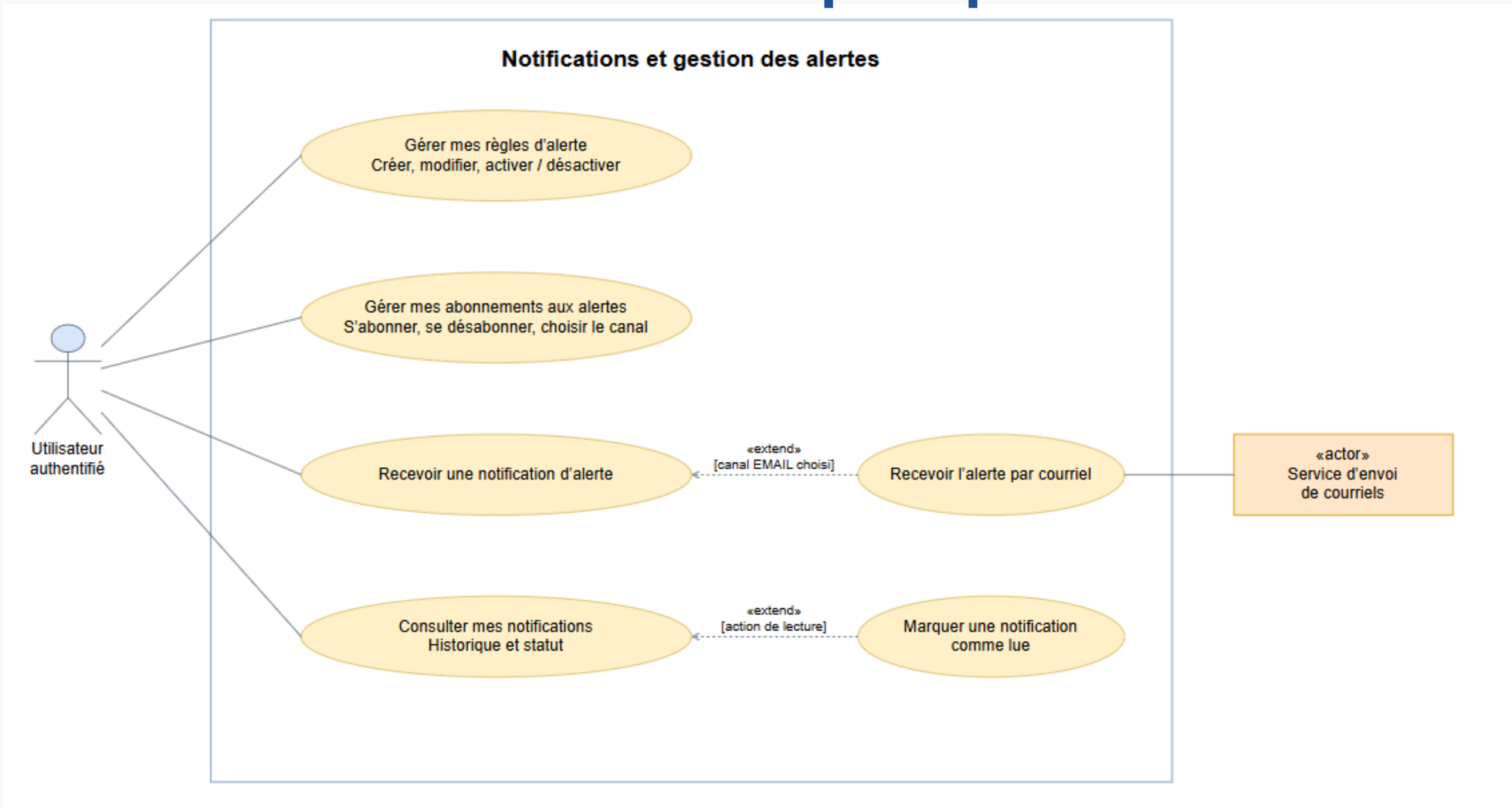


Objectif

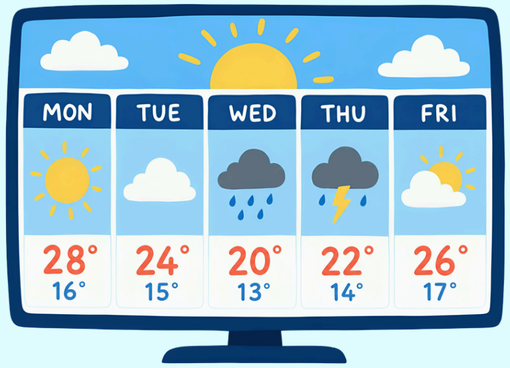
Permettre à l'utilisateur de recevoir des notifications pour les villes qu'il a sélectionnées, selon ses règles d'alerte (pluie prévue, température supérieure ou inférieure à un seuil, etc.).

MS5 — Notifications et gestions des alertes

Fonctionnalités principales



MS6 — Air Quality & UV

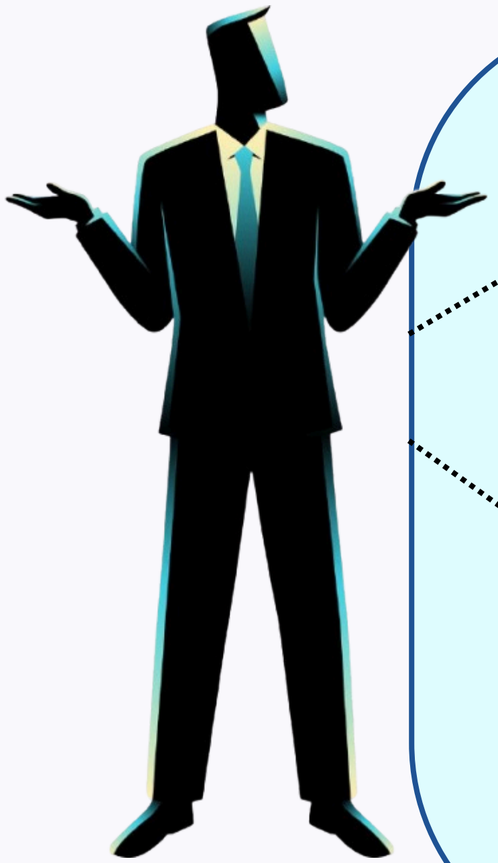


Objectif

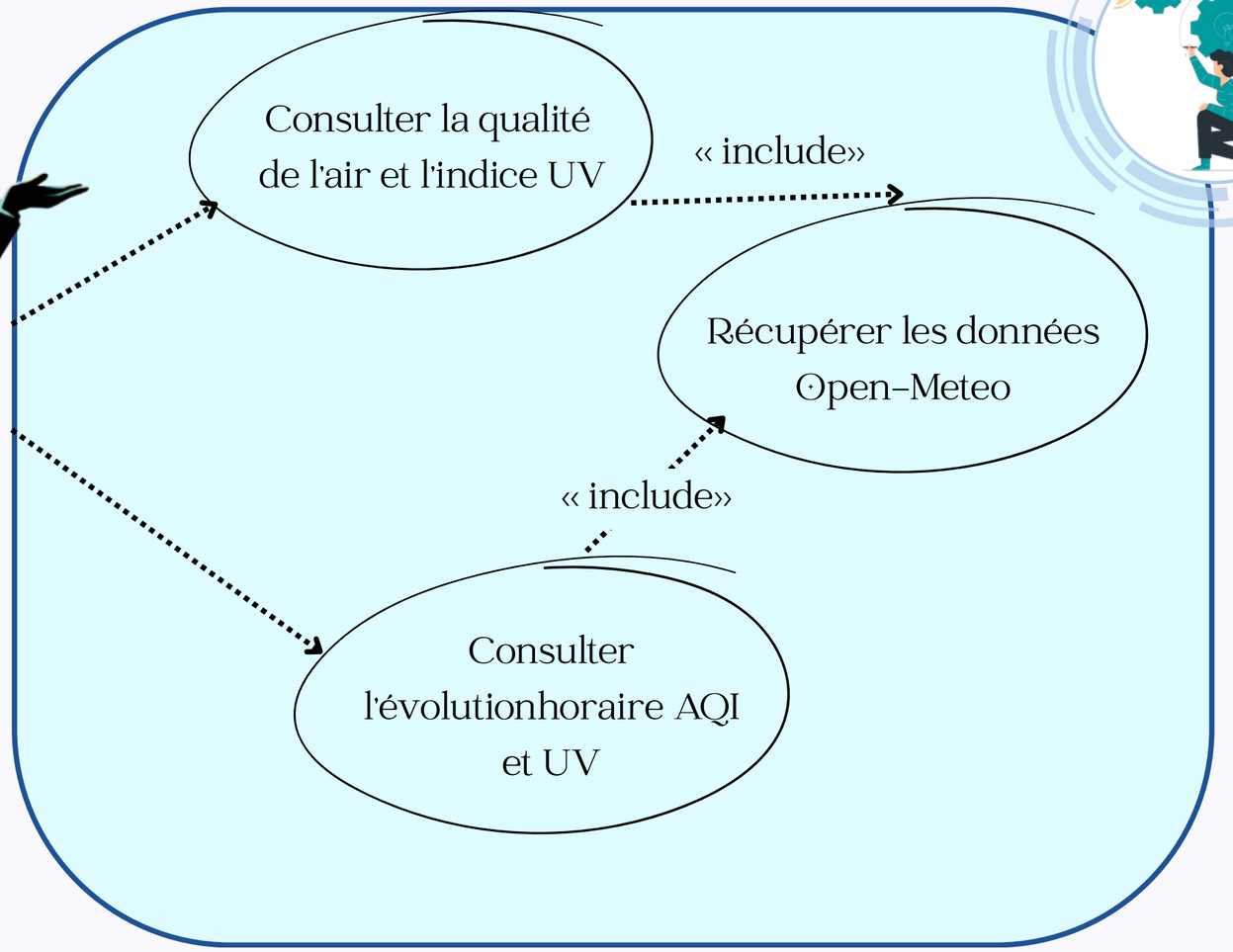
Permettre aux utilisateurs de consulter la qualité de l’air (AQI, PM2.5, PM10) et l’indice UV d’une ville, en récupérant les données depuis Open-Meteo et en stockant les relevés horaires dans InfluxDB pour suivre leur évolution.

MS6 — Air Quality & UV

Fonctionnalités principales



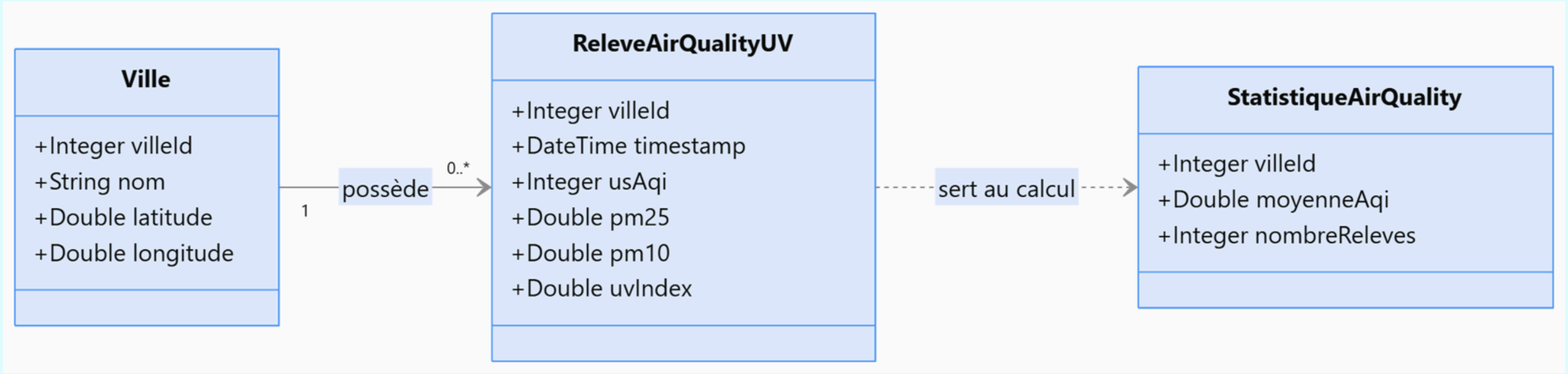




MS6 — Air Quality & UV



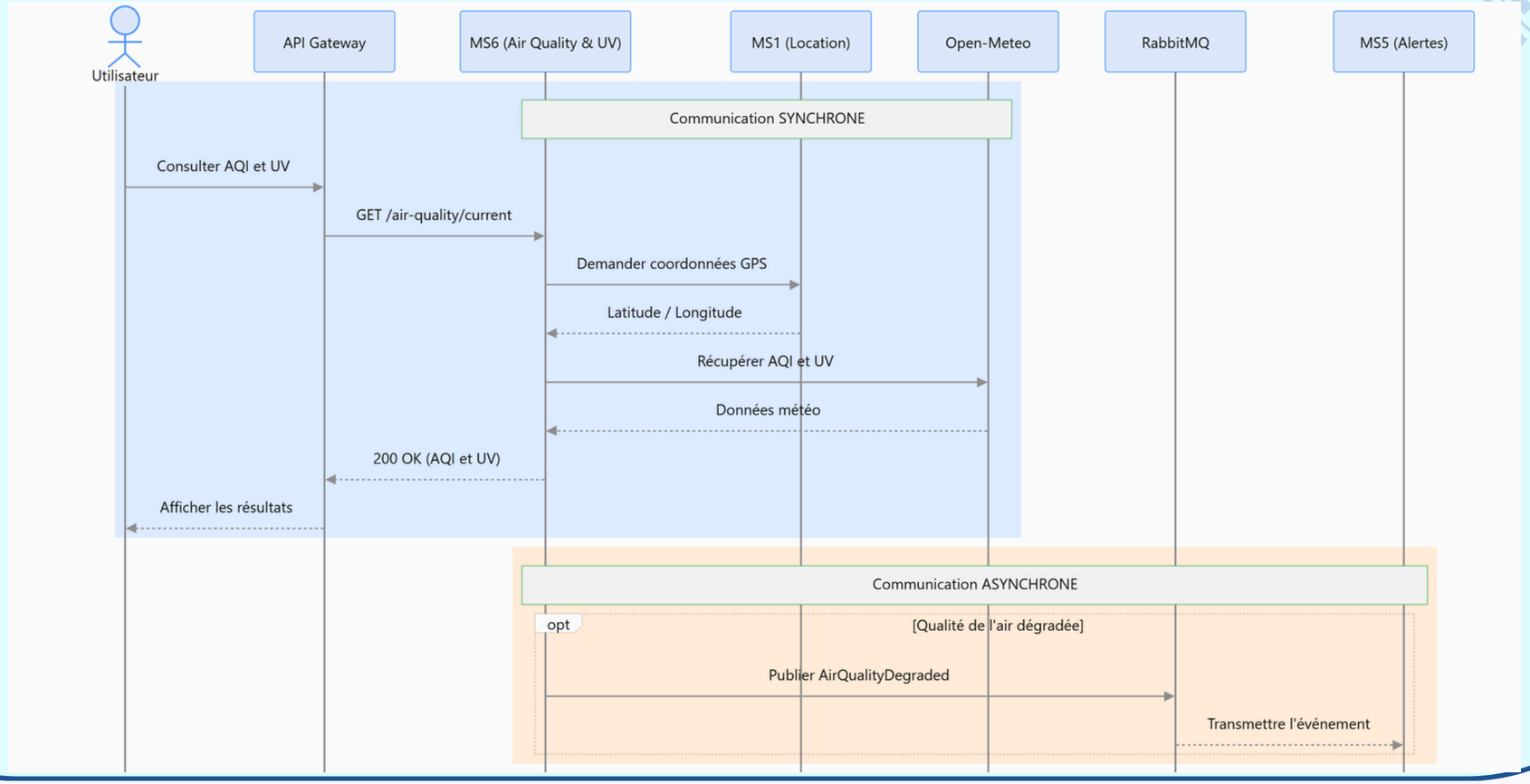
Conception



MS6 — Air Quality & UV



Communication Synchrone et asynchrone



MS6 — Air Quality & UV



Principaux endpoints

- GET /api/air-quality/current/{villeId}
- GET /api/air-quality/current/{villeId}
- GET /api/air-quality/history/{villeId}
- POST /api/air-quality/sync/{villeId}

Contrat d'API — MS6 Air Quality & UV

Définition des endpoints, formats, paramètres, authentification et erreurs.

Endpoints

GET `/api/air-quality/current/{villeId}`
Consulte la qualité de l'air et l'indice UV actuels.

GET `/api/air-quality/history/{villeId}`
Récupère les relevés horaires (AQI, PM2.5, PM10, UV).

GET `/api/air-quality/average/{villeId}`
Calcule la moyenne AQI sur les dernières 24 h.

POST `/api/air-quality/sync/{villeId}`
Récupère les données Open-Meteo et les enregistre dans InfluxDB.

Détails de l'API

Rôle	Fournir la qualité de l'air (AQI, PM2.5, PM10) et l'indice UV.
Authentification	GET : publique (sans authentification) POST /sync : JWT requis (rôle technique)
Paramètres	`villeId : Integer` (identifiant de la ville – MS1)
Source de données	Open-Meteo (récupération) + InfluxDB (stockage)
Cache	Réutilisation des relevés tant que la dernière synchronisation est récente.

Exemple de requête

GET `/api/air-quality/current/1`

Réponse 200 OK

JSON

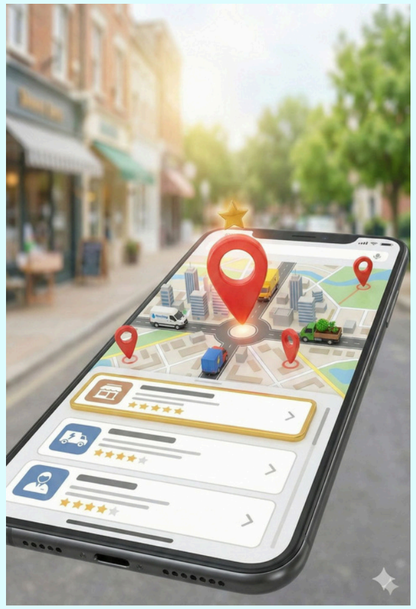
```
{
  "villeId": 1,
  "timestamp": "2026-11-08T15:[illegible]01:00",
  "usAqi": 54,
  "pm25": 8.3,
  "pm10": 13.8,
  "uvIndex": 0.95
}
```

Structure des erreurs

```
{
  "status": 404,
  "message": "Ville inconnue",
  "code": "CITY_NOT_FOUND"
}
```

400	404	503
Identifiant invalide	Ville inconnue	Source indisponible

MS7 — Service de gestion de l'historique météorologique et d'analyse climatique.





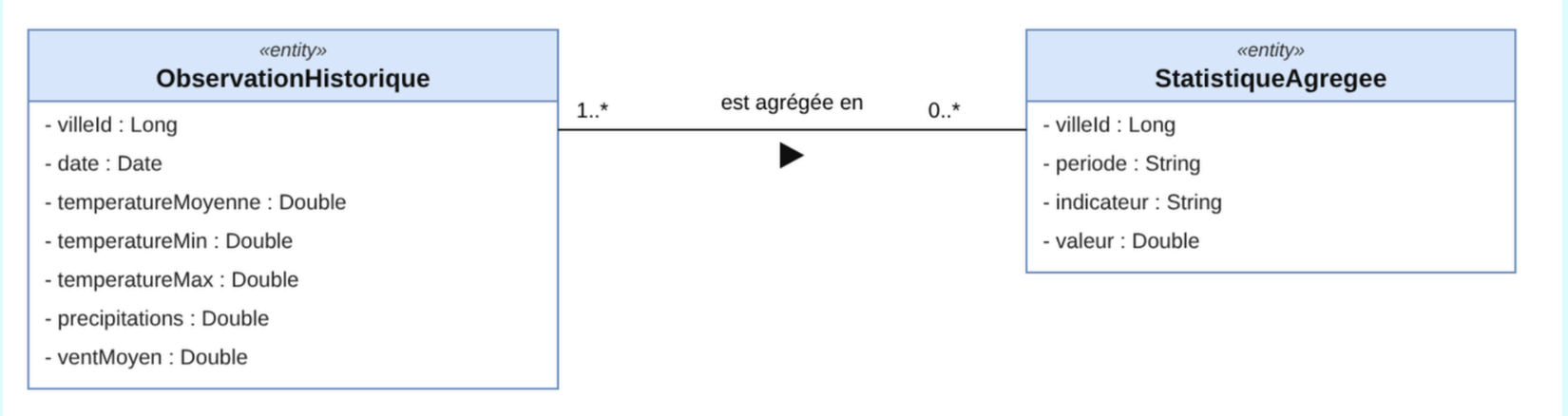
Objectif

Conserver l’historique météo et permettre des analyses climatiques. Les données sont stockées dans Cassandra.

MS7 — Service de gestion de l'historique météorologique et d'analyse climatique.



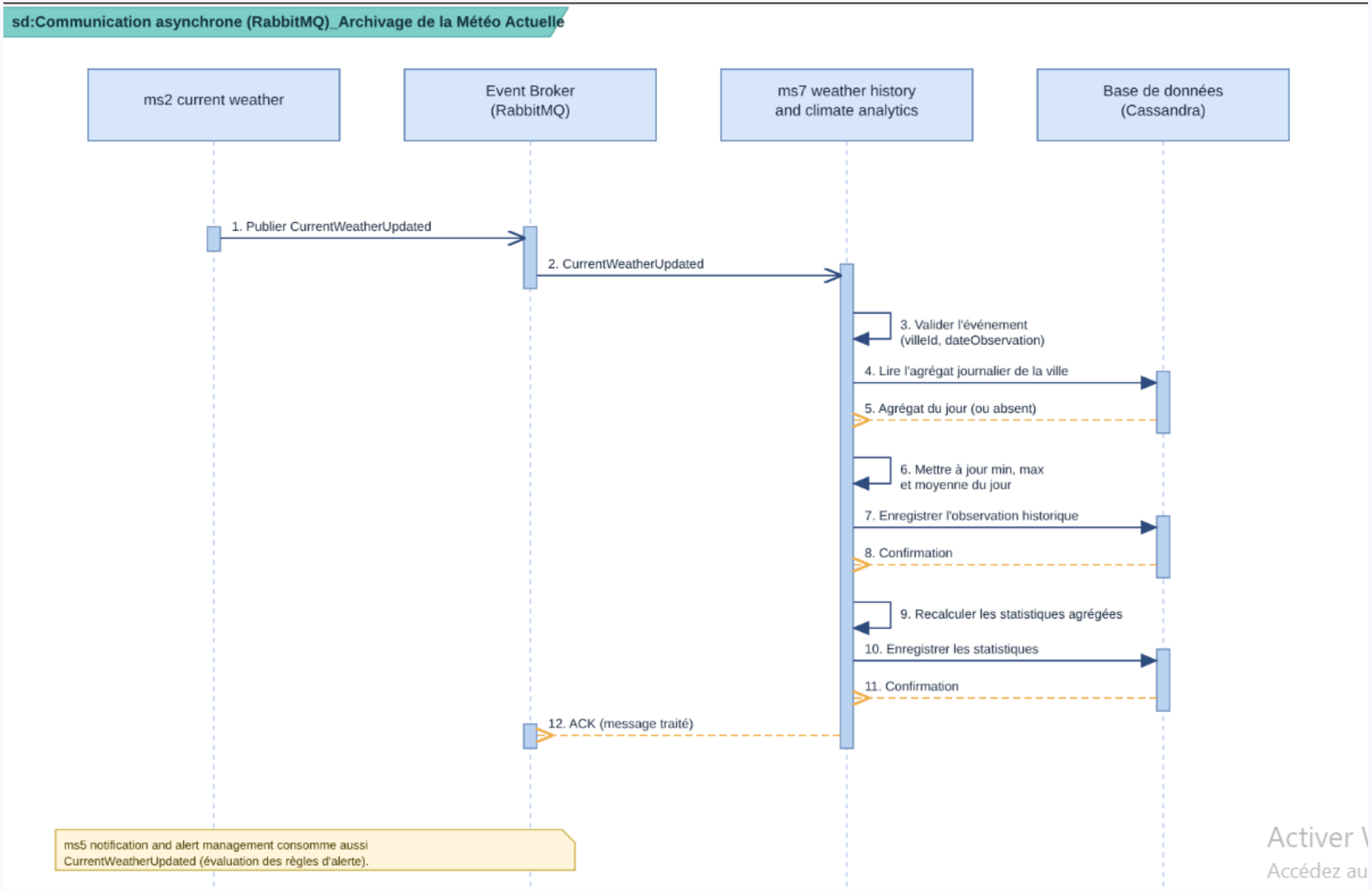
Conception

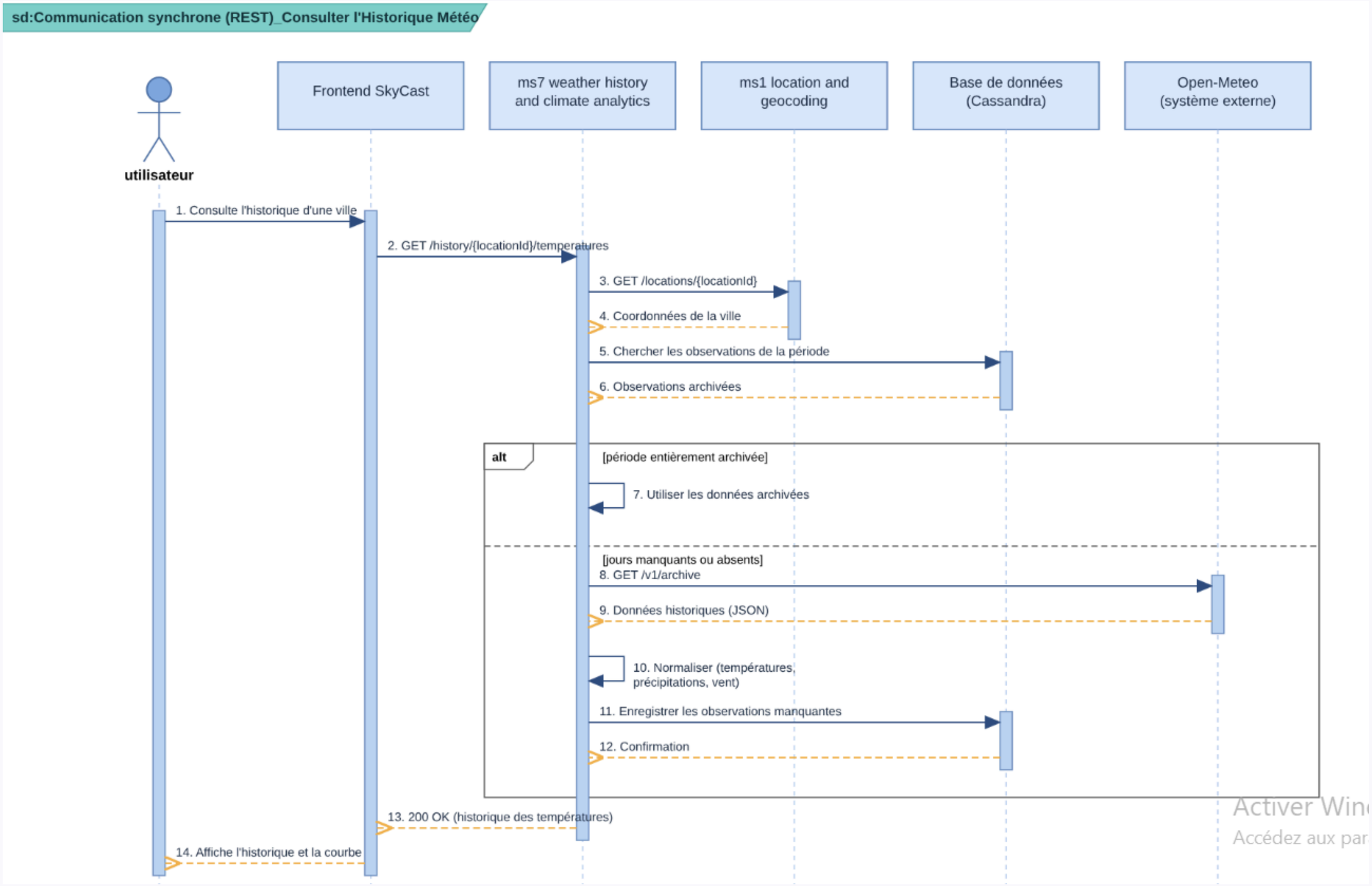


MS7 — Service de gestion de l'historique météorologique et d'analyse climatique.

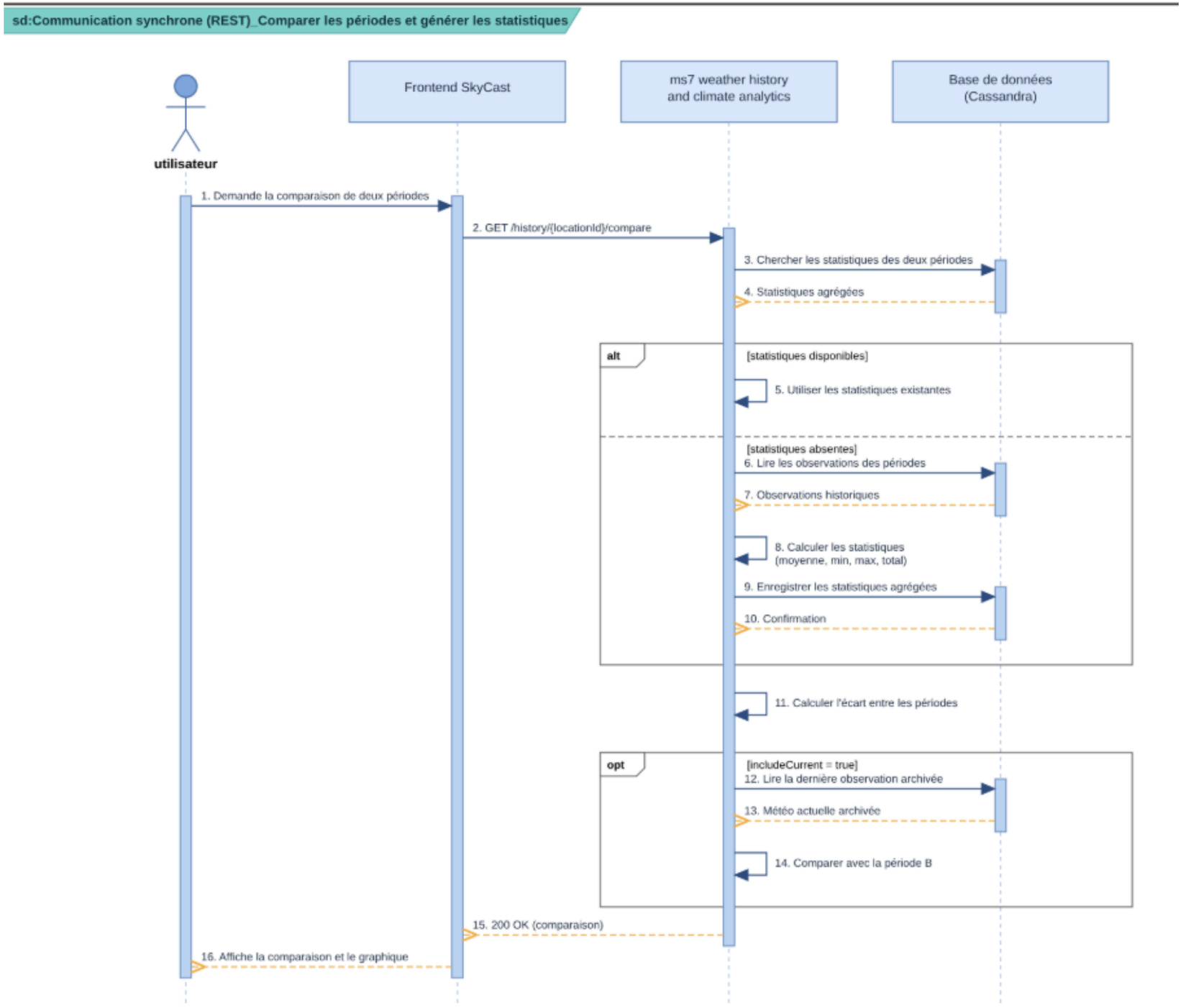


Communication Synchrone et asynchrone


sd:Communication asynchrone (RabbitMQ)_Archivage de la Météo Actuelle
ms2 current weather
Event Broker (RabbitMQ)
ms7 weather history and climate analytics
Base de données (Cassandra)
1. Publier CurrentWeatherUpdated
2. CurrentWeatherUpdated
3. Valider l'événement (villeId, dateObservation)
4. Lire l'agrégat journalier de la ville
5. Agrégat du jour (ou absent)
6. Mettre à jour min, max et moyenne du jour
7. Enregistrer l'observation historique
8. Confirmation
9. Recalculer les statistiques agrégées
10. Enregistrer les statistiques
11. Confirmation
12. ACK (message traité)
ms5 notification and alert management consomme aussi CurrentWeatherUpdated (évaluation des règles d'alerte).



Activer Win
Accédez aux par


sd:Communication synchrone (REST)_Comparer les périodes et générer les statistiques
utilisateur
Frontend SkyCast
ms7 weather history and climate analytics
Base de données (Cassandra)
1. Demande la comparaison de deux périodes
2. GET /history/{locationId}/compare
3. Chercher les statistiques des deux périodes
4. Statistiques agrégées
alt
[statistiques disponibles]
5. Utiliser les statistiques existantes
[statistiques absentes]
6. Lire les observations des périodes
7. Observations historiques
8. Calculer les statistiques (moyenne, min, max, total)
9. Enregistrer les statistiques agrégées
10. Confirmation
11. Calculer l'écart entre les périodes
opt
[includeCurrent = true]
12. Lire la dernière observation archivée
13. Météo actuelle archivée
14. Comparer avec la période B
15. 200 OK (comparaison)
16. Affiche la comparaison et le graphique

MS7 — Endpoints REST proposés

Weather History & Climate Analytics Service | API publique via la passerelle

Consultation de l'historique, agrégation des mesures et comparaison climatique

GET	`/api/v1/history/cities/{villeId}/observations`	F7.1–F7.2 • Historique des températures et évolution sur une période
GET	`/api/v1/history/cities/{villeId}/precipitation-wind`	F7.4 • Historique des précipitations et du vent
GET	`/api/v1/history/cities/{villeId}/statistics`	F7.5 • Statistiques agrégées pour alimenter les graphiques
POST	`/api/v1/history/comparisons`	F7.3 • Comparaison de deux périodes pour une ville
GET	`/api/v1/history/cities/{villeId}/current-comparison`	F7.6 • Comparaison de l'historique avec les conditions actuelles

Convention : {villeId} = identifiant logique fourni par MS1. Les dates sont au format YYYY-MM-DD. Les routes ci-dessus sont une proposition de contrat à valider avec l'implémentation.

Activer Windows

MS7 – Exemples de contrats d'API

Paramètres d'entrée et formats JSON des principales opérations

1. Consulter l'historique

`GET /api/v1/history/cities/{villeId}/observations`

Paramètres de requête

```
startDate  = 2026-01-01
endDate    = 2026-01-31
```

Réponse – 200 OK


```
{
  "villeId": 42,
  "observations": [
    {
      "date": "2026-01-01",
      "temperatureMoyenne": 14.2,
      "temperatureMin": 9.1,
      "temperatureMax": 18.7,
      "precipitations": 2.4,
      "ventMoyen": 13.5
    }
  ]
}
```


Exemples de structures proposées – à valider avec l'implémentation du service.

2. Comparer deux périodes

`POST /api/v1/history/comparisons`

Corps de la requête (JSON)


```
{
  "villeId": 42,
  "periodeA": {
    "startDate": "2025-01-01",
    "endDate": "2025-01-31"
  },
  "periodeB": {
    "startDate": "2026-01-01",
    "endDate": "2026-01-31"
  },
  "indicateurs": [
    "temperatureMoyenne",
    "precipitations"
  ]
}
```


Réponse – 200 OK


```
{ "villeId": 42, "comparaison": [ { "indicateur": "temperatureMoyenne", "valeurA": 12.8,
"valeurB": 14.2, "ecart": 1.4 } ] }
```



5
Conclusion

MERCI POUR VOTRE ATTENTION

